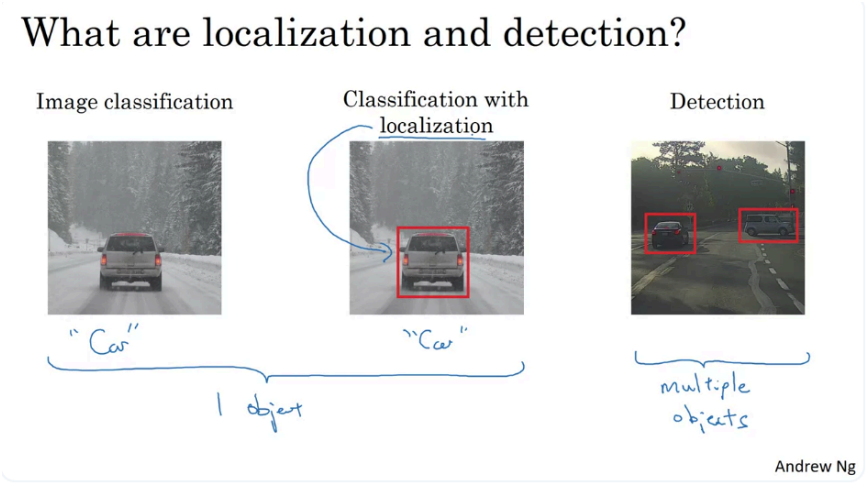
Object Detection

Object Localization



Object Detection – Introduction and Motivation

Big Picture

This week focuses on **Object Detection**, one of the fastest-advancing areas in computer vision.

Before learning object detection, it is important to understand **Object Localization**, which serves as a foundation for more advanced detection tasks.

---

1. Image Classification

Goal

Determine **what object** is present in an image.

Example

Input:

- Image containing a car

Output:

- "Car"

Characteristics

- Predicts only the object's category.
- Does **not** provide information about the object's location.

Visualization:

```
Image → Neural Network → Car
```

---

2. Classification with Localization

Goal

Determine:

1. What the object is.
2. Where the object is located in the image.

Example

Input:

- Image containing a car

Output:

- Label: "Car"
- Bounding box around the car

Visualization:

```
Image → Neural Network
              ↓
          Car
       [Bounding Box]
```

Key Concept: Localization

Localization means identifying the position of the detected object inside the image.

Typically represented using a **bounding box** (rectangle surrounding the object).

Difference from Classification

Task	Object Category	Object Location
Classification	Yes	No
Classification + Localization	Yes	Yes

---

3. Object Detection

Goal

Detect and localize **multiple objects** within the same image.

Example

An autonomous driving scene may contain:

- Cars
- Pedestrians
- Motorcycles
- Other road objects

The algorithm must:

- Detect every object.
- Classify each object.
- Draw a bounding box around each object.

Visualization:

```
Image
 ├── Car         → Bounding Box
 ├── Pedestrian  → Bounding Box
 └── Motorcycle  → Bounding Box
```

---

4. Classification + Localization vs Detection

Classification + Localization

Characteristics:

- Usually only one major object.
- Often centered in the image.

- Task:
  - Recognize the object.
  - Localize the object.

Example:

```
Single car image
→ Car
→ Bounding box
```

Object Detection

Characteristics:

- Multiple objects may appear.
- Objects may belong to different categories.
- Every object must be classified and localized.

Example:

```
Street image
→ Car
→ Car
→ Pedestrian
→ Motorcycle
→ Bounding boxes for all
```

Learning Progression

The course builds concepts in the following order:

```
Image Classification
        ↓
Classification + Localization
        ↓
Object Detection
```

Why this order?

- Knowledge from image classification helps solve localization.
- Knowledge from localization becomes the foundation for object detection.

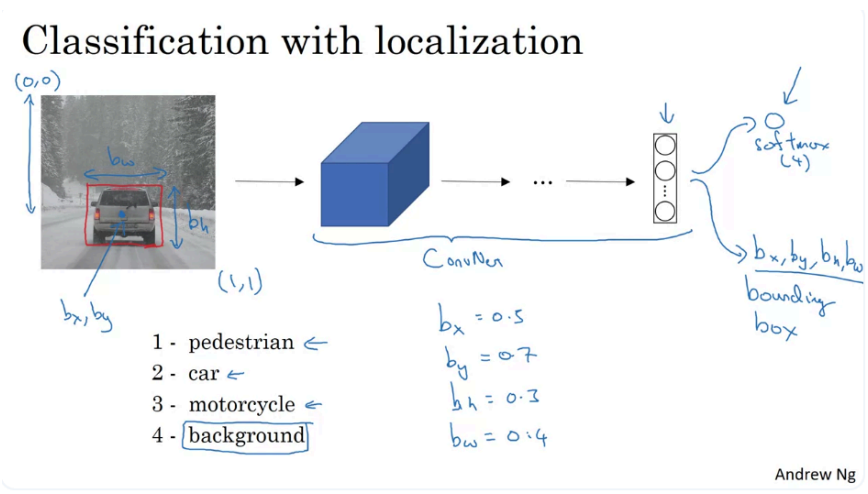
Key Takeaways

1. **Image Classification**
   - Predicts what object is present.
   - Does not provide location information.
2. **Classification with Localization**
   - Predicts the object category.
   - Predicts the object's position using a bounding box.
3. **Object Detection**
   - Handles multiple objects.
   - Classifies and localizes every object in the image.
4. **Localization**
   - Refers to finding where an object appears in an image.

5. The learning path is:

```
Classification
    → Localization
    → Detection
```

This lecture serves as the conceptual introduction to object detection and explains why localization is the crucial intermediate step between image classification and full object detection.



Classification with Localization: Predicting Bounding Boxes

Recap: Standard Image Classification

In a traditional image classification system:

```
Image
  ↓
ConvNet
  ↓
Feature Vector
  ↓
Softmax
  ↓
Class Prediction
```

For a self-driving car application, possible classes might be:

Class	Meaning
Pedestrian	Human walking
Car	Vehicle
Motorcycle	Motorcycle
Background	None of the above

The softmax layer outputs probabilities over these four classes.

Example:

```
[Pedestrian, Car, Motorcycle, Background]
[  0.02   , 0.95,    0.02    ,   0.01   ]
```

Prediction: **Car**

Limitation

Classification only answers:

> "What object is in the image?"

It does not answer:

> "Where is the object?"

Extending the Network for Localization

To localize an object, the network must predict a **bounding box** in addition to the class label.

The output layer is extended as follows:

```
Class probabilities
      +
Bounding box parameters
```

New outputs:

```
bx, by, bh, bw
```

where:

Variable	Meaning
bx	x-coordinate of bounding box center
by	y-coordinate of bounding box center
bh	height of bounding box
bw	width of bounding box

Bounding Box Coordinate System

The lecture uses a normalized coordinate system:

```
(0,0) -----------------> x
  |
  |
  |
  v
  y
```

- Top-left corner = (0,0)
- Bottom-right corner = (1,1)

This normalization makes the representation independent of image size.

Bounding Box Representation

Instead of storing the four corners of the box, the bounding box is represented by:

1. Center point `(bx, by)`
2. Height `bh`
3. Width `bw`

Visualization:

```
      +----------------+
      |                |
      |       •        |
      |    (bx,by)     |
      |                |
      +----------------+

Height = bh
Width  = bw
```

Example

Suppose a car appears approximately in the center-lower part of the image.

Estimated values:

Parameter	Value	Interpretation
bx	0.5	Center is halfway across image width
by	0.7	Center is 70% down image height
bh	0.3	Bounding box height is 30% of image height
bw	0.4	Bounding box width is 40% of image width

Thus:

```
(bx, by, bh, bw)
=
(0.5, 0.7, 0.3, 0.4)
```

New Neural Network Output

The network now predicts both:

Object Class

```
Pedestrian / Car / Motorcycle / Background
```

Bounding Box

```
bx, by, bh, bw
```

Complete output:

```
[Class Scores]
      +
[bx, by, bh, bw]
```

Example:

```
Car
bx = 0.5
by = 0.7
bh = 0.3
bw = 0.4
```

Training Data Requirements

For localization, the training set must contain:

Before (Classification)

```
Image → Class Label
```

Example:

```
Image → Car
```

After (Classification + Localization)

```
Image →
    Class Label
    Bounding Box
```

Example:

```
Image →
    Car
    (0.5, 0.7, 0.3, 0.4)
```

The bounding box annotations are usually created manually by humans.

Supervised Learning Formulation

The task becomes:

```
Input:
    Image

Target:
    Object Class
    +
    Bounding Box Parameters
```

The neural network learns to predict:

```
y =
[
class label,
bx,
by,
bh,
bw
]
```

using supervised learning.

Key Takeaways

Standard Classification

- Predicts only the object category.
- Uses a softmax output layer.
- Cannot determine object location.

Classification + Localization

- Predicts object category.
- Predicts bounding box coordinates.

Bounding Box Parameters

Parameter	Description
bx	Center x-coordinate
by	Center y-coordinate
bh	Box height
bw	Box width

Coordinate Convention

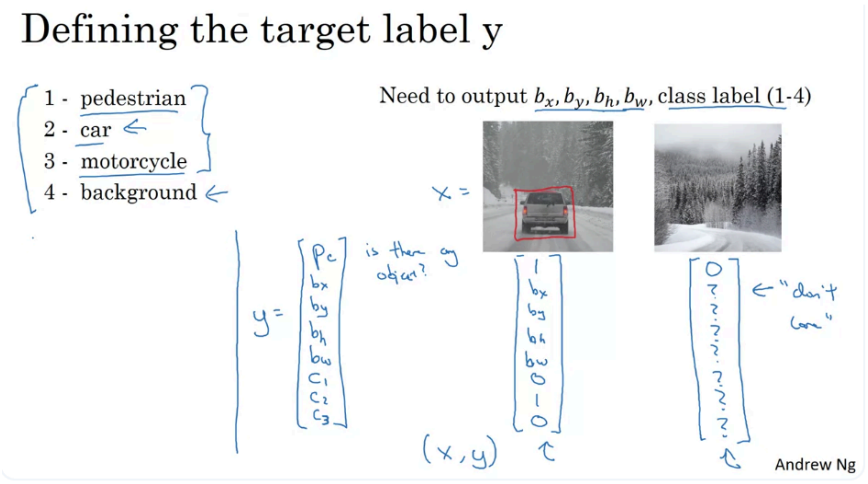
- Top-left = (0,0)
- Bottom-right = (1,1)
- Coordinates are normalized to the image dimensions.

Training Requirement

Each training image must include:

```
Class Label
+
Bounding Box Annotation
```

so the network can learn both recognition and localization simultaneously.



Classification with Localization: Label Representation and Loss Function

Learning Objective

Extend image classification so that the network can simultaneously:

1. Determine whether an object exists.
2. Predict the object's location (bounding box).
3. Predict the object's category.

This lecture introduces the **target label format** and the **loss function** used to train such a network.

---

1. Output Representation

Previously, the network only predicted object classes:

Class	Meaning
Class 1	Pedestrian
Class 2	Car
Class 3	Motorcycle
Background	No target object

For localization, the network now predicts:

```
[pc, bx, by, bh, bw, c1, c2, c3]
```

Total: **8 outputs**.

---

2. Meaning of Each Output

Object Presence Indicator (`pc`)

`pc` answers:

> "Is there a target object in the image?"

```
pc = 1  → object exists
pc = 0  → no object (background)
```

Interpretation:

- `pc` can be viewed as the probability that one of the target classes exists.
- Background means none of the target classes are present.

Bounding Box Coordinates

If an object exists (`pc = 1`), the network predicts:

```
bx, by, bh, bw
```

where:

Parameter	Meaning
bx	center x-coordinate
by	center y-coordinate
bh	bounding box height
bw	bounding box width

These define the object's location.

Class Indicators

The final three outputs identify the object category:

```
[c1, c2, c3]
```

Variable	Class
c1	Pedestrian
c2	Car
c3	Motorcycle

Since the image contains at most one object:

```
Only one of c1, c2, c3 can be 1.
```

Example:

```
Car
→ c1 = 0
→ c2 = 1
→ c3 = 0
```

---

3. Target Label Example: Object Present

Suppose the image contains a car.

Target label:

```
y =
[
1,
bx,
by,
bh,
bw,
0,
1,
0
]
```

Explanation:

Component	Value	Meaning
pc	1	Object exists
bx,by,bh,bw	Bounding box coordinates	Location
c1	0	Not pedestrian
c2	1	Car
c3	0	Not motorcycle

Training data must therefore include manually annotated bounding boxes.

---

4. Target Label Example: No Object Present

Suppose the image contains only background.

Target label:

```
y =
[
0,
?,
?,
?,
?,
?,
?,
?
]
```

where:

```
? = don't care
```

Reason:

If no object exists:

- Bounding box is irrelevant.
- Object category is irrelevant.

The network is only required to learn:

```
pc = 0
```

Everything else is ignored.

---

5. Why Use "Don't Care" Values?

When the image contains no object:

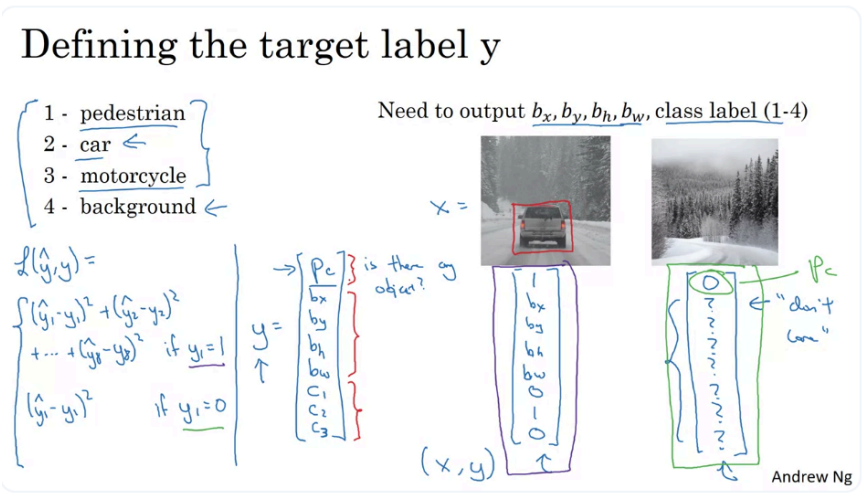
```
No object
→ No bounding box needed
→ No class label needed
```

Therefore:

```
bx, by, bh, bw, c1, c2, c3
```

should not contribute to the loss.

Only the object-existence prediction matters.



6. Loss Function

Let:

```
y     = ground truth label
ŷ (yhat) = network prediction
```

The lecture uses squared error for simplicity.

Case 1: Object Exists (`pc = 1`)

Target:

```
y1 = pc = 1
```

Loss:

$$L = \sum_{i=1}^{8} (\hat{y}_i - y_i)^2$$

Meaning:

Penalize errors in:

- Object existence prediction
- Bounding box coordinates
- Object class prediction

Every component contributes to the loss.

Case 2: No Object Exists (`pc = 0`)

Target:

```
y1 = pc = 0
```

Loss:

$L = (\hat{y}_1 - y_1)^2$

Only the object-presence output is evaluated.

All remaining outputs are ignored because they correspond to "don't care" values.

---

7. Practical Loss Functions

The instructor notes that squared error is used only to simplify the explanation.

In practice:

Output Type	Common Loss
`pc`	Logistic/Binary Cross Entropy
`c1,c2,c3`	Softmax Cross Entropy
`bx,by,bh,bw`	Squared Error (L2) or Smooth L1

A real object detection model typically combines classification and regression losses.

Complete Pipeline

```
Input Image
      ↓
   ConvNet
      ↓
[pc, bx, by, bh, bw, c1, c2, c3]
```

Where:

```
pc            → object exists?
bx,by,bh,bw   → object location
c1,c2,c3      → object category
```

This is one of the earliest examples of a neural network performing both:

- Classification
- Regression

simultaneously. The key idea is that the network outputs not only class probabilities but also real-valued coordinates describing where the object is located in the image.

Key Takeaways

1. The output label is:

```
[pc, bx, by, bh, bw, c1, c2, c3]
```

1. `pc` indicates whether an object exists.
2. Bounding box localization is represented by:

```
bx, by, bh, bw
```

1. Object class is represented by:

```
c1, c2, c3
```

1. When no object exists:

```
pc = 0
```

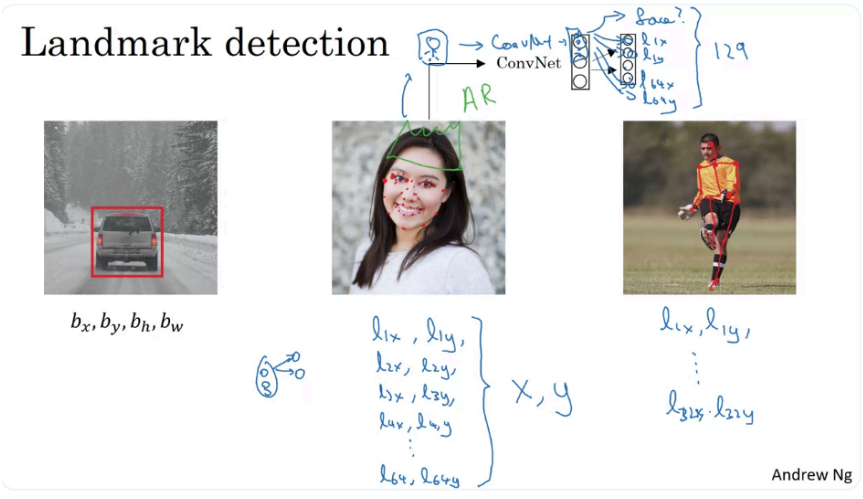
and all remaining fields become "don't care".

1. Loss function depends on whether an object is present:

- Object present → evaluate all outputs.
- No object → evaluate only `pc`.

1. This formulation turns object localization into a joint **classification + regression** problem, which becomes the foundation for modern object detection methods.

Landmark Detection



Motivation

In the previous lecture, the neural network predicted:

```
bx, by, bh, bw
```

to localize an object using a bounding box.

A more general idea is to have the neural network directly predict the coordinates of important points in an image, called **landmarks**.

Instead of predicting a rectangle, the network predicts the locations of specific key points.

---

1. What is a Landmark?

A landmark is a predefined point of interest in an image.

Examples:

- Corner of an eye
- Tip of the nose
- Corner of the mouth
- Shoulder joint
- Elbow joint

Each landmark is represented by:

```
(lx, ly)
```

where:

- `lx` = x-coordinate
- `ly` = y-coordinate

The network learns to predict these coordinates directly.

---

2. Single Landmark Detection

Suppose we only care about one point:

```
Corner of the eye
```

Network architecture:

```
Face Image
     ↓
  ConvNet
     ↓
  lx, ly
```

Output:

```
(lx, ly)
```

which specifies the location of that eye corner.

---

3. Multiple Landmark Detection

Suppose we want to detect the four eye corners.

Define:

```
Landmark 1
Landmark 2
Landmark 3
Landmark 4
```

The network outputs:

```
l1x, l1y
l2x, l2y
l3x, l3y
l4x, l4y
```

Architecture:

```
Face Image
     ↓
  ConvNet
     ↓
[l1x,l1y,
 l2x,l2y,
 l3x,l3y,
 l4x,l4y]
```

Now the network predicts all four landmark positions simultaneously.

---

4. Dense Facial Landmark Detection

The idea scales beyond a few points.

Possible landmarks:

Eyes

```
Eye corners
Upper eyelid points
Lower eyelid points
```

Mouth

```
Mouth corners
Upper lip points
Lower lip points
```

Nose

```
Nose bridge
Nose edges
Nose tip
```

Face Shape

```
Jawline
Face contour
```

The lecture gives an example of using:

```
64 landmarks
```

across the face.

---

5. Neural Network Output for 64 Landmarks

For each landmark:

```
(x,y)
```

Two numbers are required.

For 64 landmarks:

```
64 × 2 = 128
```

coordinates.

The network may also predict whether a face exists:

```
pc
```

Therefore total outputs become:

```
1 + 128 = 129
```

outputs.

Output Vector

```
[
pc,
l1x,
l1y,
l2x,
l2y,
...
l64x,
l64y
]
```

where:

```
pc = face present?
```

and

```
li = landmark i
```

6. Applications of Facial Landmark Detection

Emotion Recognition

Landmarks capture facial geometry.

Example:

Smiling

```
Mouth corners move upward
```

Frowning

```
Mouth corners move downward
```

The landmark positions can be used to classify emotions.

Augmented Reality (AR)

Examples:

- Snapchat filters
- Instagram filters
- Face masks
- Virtual glasses
- Virtual crowns
- Virtual hats

Once facial landmarks are known, graphics can be accurately attached to the face.

Face Warping and Graphics Effects

Landmarks define facial geometry.

This enables:

```
Face deformation
Face morphing
```

```
Beauty filters
Animation effects
```

because the system knows exactly where key facial structures are located.

---

7. Training Data Requirements

Landmark detection requires supervised learning.

Training data contains:

```
Image
+
Landmark coordinates
```

Example:

```
Image A
→ (l1x,l1y)
→ (l2x,l2y)
...
→ (l64x,l64y)
```

These landmarks are typically annotated manually by humans.

The lecture emphasizes that creating such datasets is labor-intensive because every landmark must be labeled.

---

8. Human Pose Estimation

The same idea can be applied to the human body.

Instead of facial landmarks, define body landmarks.

Examples:

```
Chest center
Left shoulder
Right shoulder
Left elbow
Right elbow
Left wrist
Right wrist
...
```

Network:

```
Person Image
      ↓
   ConvNet
      ↓
Body Landmark Coordinates
```

The predicted landmarks describe the person's pose.

Example Output

If 32 body landmarks are used:

```
l1x,l1y
...
l32x,l32y
```

The network outputs:

```
64 coordinate values
```

that define the skeleton of the person.

---

9. Critical Requirement: Landmark Consistency

A very important requirement is:

> The meaning of each landmark must remain the same across all images.

Example:

```
Landmark 1 = left eye corner
Landmark 2 = right eye corner
Landmark 3 = nose tip
...
```

This definition must never change.

Incorrect:

```
Image A:
Landmark 1 = left eye

Image B:
Landmark 1 = nose tip
```

Such inconsistency would make learning impossible.

The network can only learn if landmark identities are consistently defined throughout the dataset.

Connection to Object Detection

The key insight of this lecture is:

> A neural network can output real-valued coordinates directly.

Bounding box prediction was one example:

```
bx, by, bh, bw
```

Landmark detection generalizes this idea:

```
l1x,l1y,
l2x,l2y,
...
lnx,lny
```

This transforms localization into a regression problem and becomes an important building block for:

- Face analysis
- Emotion recognition
- Augmented reality
- Human pose estimation
- Object detection

The next step in the course is to use these localization ideas to build full object detection systems.

Key Takeaways

1. A **landmark** is a predefined key point in an image.

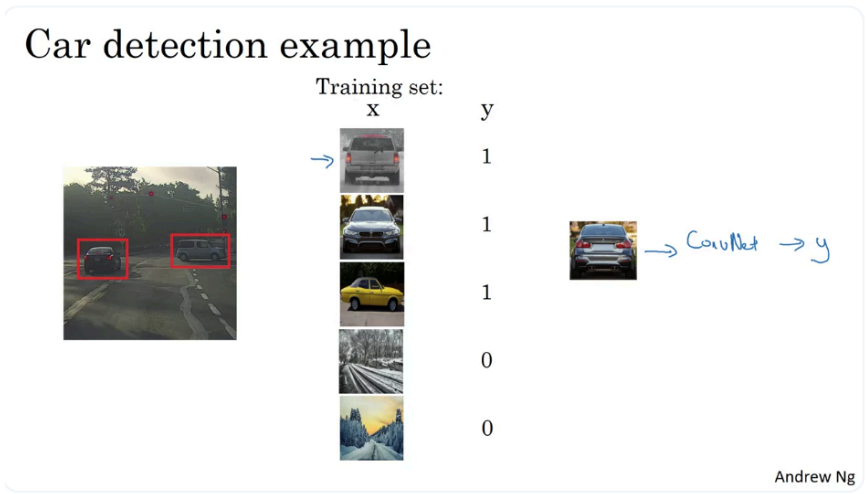
2. Landmarks are represented as:

```
(x,y)
```

coordinates.

1. Neural networks can predict many landmarks simultaneously.
2. Facial landmarks enable:
    - Emotion recognition
    - Face tracking
    - AR filters
    - Face warping
3. Human pose estimation is another landmark detection problem.
4. Landmark labels must be consistent across all training images.
5. Landmark detection is a generalization of bounding box localization and serves as an important foundation for modern computer vision systems.

Object Detection



Sliding Windows Detection Algorithm

Motivation

Object localization assumes:

- One object in the image.
- The object occupies a significant portion of the image.

Real-world object detection is harder because:

- Objects can appear anywhere.
- Objects can have different sizes.
- Multiple objects may exist.

One of the earliest solutions is the **Sliding Windows Detection Algorithm**.

Step 1: Train a Binary Classifier

Create a Training Dataset

Construct a dataset containing:

```
(x, y)
```

where:

- `x` = image patch
- `y` = car / not car

Example:

Image Patch	Label
Cropped car	1
Cropped car	1
Cropped car	1
Background	0
Background	0

The positive examples should be **tightly cropped around the car**.

```
+-----------+
|           |
|    CAR    |
|           |
+-----------+
```

The car should occupy most of the image.

The goal is to teach the network:

> Does this image patch contain a car?

not

> Where is the car?

Train the ConvNet

Network:

```
Image Patch
     ↓
```

```
  ConvNet
     ↓
Car / Not Car
```

Output:

```
1 → Car
0 → Not Car
```

After training, the ConvNet becomes a car classifier.

Step 2: Apply Sliding Windows on a Large Image

Suppose we have a test image:

```
+----------------------+
|                      |
|      Car Here        |
|                      |
+----------------------+
```

The ConvNet expects a small cropped image as input.

Therefore, we scan the image using a window.

First Window

Choose a window size.

```
+----------------------+
|[ ]                   |
|                      |
|                      |
+----------------------+
```

Crop the region inside the window.

Feed it into the ConvNet:

```
Window Patch
      ↓
   ConvNet
      ↓
Car / Not Car
```

Prediction:

```
0
```

because this region does not contain a car.

Slide the Window

Move the window slightly.

```
+----------------------+
|   [ ]                |
|                      |
|                      |
+----------------------+
```

Run the ConvNet again.

Repeat:

```
Crop
 ↓
ConvNet
 ↓
Prediction
```

for every location in the image.

Stride

The amount the window moves each step is called the **stride**.

Example:

```
Position 1
     ↓
Position 2
     ↓
Position 3
```

A larger stride:

```
Move 20 pixels
```

Advantages:

- Faster
- Fewer evaluations

Disadvantages:

- May miss objects
- Poor localization

A smaller stride:

```
Move 2 pixels
```

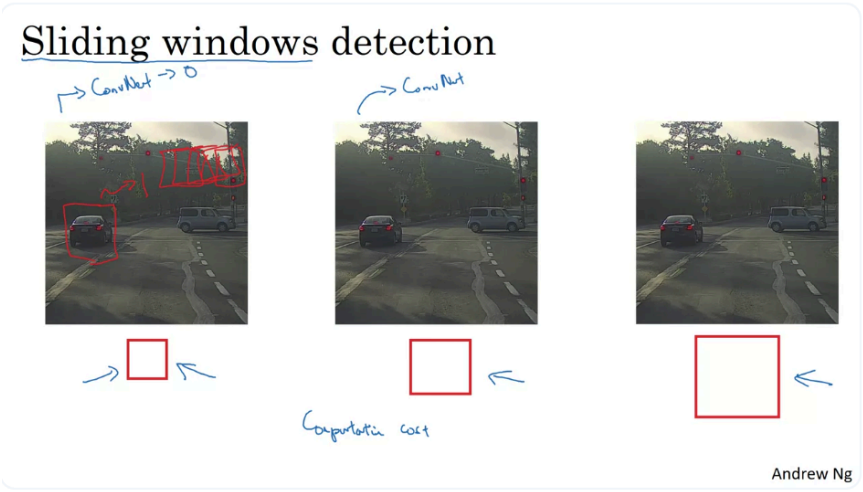
Advantages:

- More accurate detection
- Better localization

Disadvantages:

- Much more computation

Detecting Different Object Sizes



A single window size is insufficient.

Why?

Because cars can appear:

```
Small
Medium
Large
```

inside the image.

Second Pass

Use a larger window.

```
+----------------------+
| [     ]              |
|                       |
|                       |
+----------------------+
```

Slide this larger window across the entire image.

Third Pass

Use an even larger window.

```
+----------------------+
|[          ]          |
|                       |
|                       |
+----------------------+
```

Again scan the entire image.

Multi-Scale Detection

The complete algorithm:

```
Small Window
      ↓
Scan Entire Image

Medium Window
      ↓
Scan Entire Image

Large Window
      ↓
Scan Entire Image
```

The hope is that for some:

```
Window Size
+
Window Position
```

the car will fit nicely inside the window.

At that moment:

```
Window Patch
      ↓
ConvNet
      ↓
1 (Car)
```

and the system detects a car.

Why It Is Called Sliding Windows

Because the detector literally takes a window and slides it across the image.

```
[ ] → [ ] → [ ] → [ ]
```

At every location:

```
Crop Patch
      ↓
Run ConvNet
      ↓
Classify
```

The image is exhaustively searched.

Major Drawback: Computational Cost

The biggest problem is efficiency.

Suppose:

```
100 × 100 positions
```

and

```
3 window sizes
```

Then:

```
30,000 ConvNet evaluations
```

may be required.

Each evaluation requires a forward pass through the network.

This becomes extremely expensive.

Stride Trade-Off

Large Stride

```
Fast
Less Computation
```

but

```
Poor Localization
May Miss Objects
```

Small Stride

```
Better Detection
Better Localization
```

but

```
Very Expensive
Many ConvNet Evaluations
```

Historical Context

Before deep learning:

- Object detectors often used hand-crafted features.
- Classifiers were simple linear models.

Because the classifier was cheap:

```
Sliding Windows
+
Linear Classifier
```

was practical.

With ConvNets:

```
Sliding Windows
+
Deep Network
```

becomes much slower.

Running thousands of ConvNet evaluations on a single image is often impractical.

Main Limitation

The algorithm suffers from:

1. Huge computational cost.
2. Many repeated computations.
3. Slow inference.
4. Poor scalability to large images.

Key Insight Leading to Modern Detectors

Although the naive sliding-window approach is slow, the core idea is still valuable:

> Search for objects by evaluating many regions of an image.

The next improvement is:

> Implement Sliding Windows Convolutionally

so that overlapping windows share computation instead of running a separate ConvNet for every crop.

This idea becomes the bridge to modern object detection methods.

Key Takeaways

Training Phase

```
Cropped Car Images
        ↓
     ConvNet
        ↓
Car / Not Car
```

Detection Phase

```
Large Image
      ↓
Sliding Windows
      ↓
Crop Patch
      ↓
ConvNet
      ↓
Car / Not Car
```

Important Concepts

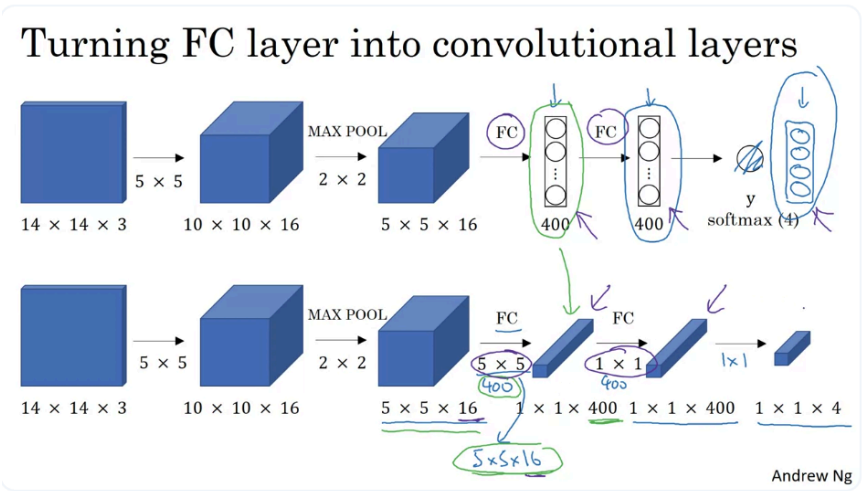
- Window = image region being evaluated.
- Stride = step size when moving the window.
- Multiple window sizes are needed for different object scales.
- Detection succeeds when one window tightly covers the object.

Main Weakness

```
Too many ConvNet evaluations
→ Very high computational cost
```

This computational bottleneck motivates the convolutional implementation introduced in the next lecture.

Convolutional Implementation of Sliding Windows



Convolutional Implementation of Sliding Windows

Motivation

In the previous lecture, we learned the **Sliding Windows Detection Algorithm**.

The main idea was:

```
Crop Window
     ↓
Run ConvNet
     ↓
Car / Not Car
```

for every possible location and scale in the image.

However, this approach is extremely slow because the ConvNet must be run thousands of times on overlapping image regions.

This lecture introduces a much more efficient solution:

> Implement the sliding-window detector convolutionally.

To understand how this works, we first need to learn how to convert **fully connected (FC) layers into convolutional layers**.

Example Network

Suppose the detector receives:

```
14 × 14 × 3 image
```

Architecture:

```
14×14×3
    ↓
5×5 Conv (16 filters)
    ↓
10×10×16
    ↓
2×2 Max Pool
    ↓
5×5×16
    ↓
FC (400)
    ↓
FC (400)
    ↓
Softmax (4 classes)
```

Classes:

```
Pedestrian
Car
Motorcycle
Background
```

Step 1: Reinterpret the Softmax Output

Instead of viewing the final output as:

```
4 nodes
```

we view it as:

```
1 × 1 × 4 volume
```

This change allows the entire network to be expressed using convolutions.

Step 2: Convert the First Fully Connected Layer

Original FC Layer

Input:

```
5 × 5 × 16
```

Output:

```
400 units
```

Every output neuron connects to all:

```
5 × 5 × 16 = 400
```

input activations.

Equivalent Convolution

Use:

```
400 filters
```

of size:

```
5 × 5 × 16
```

Input:

```
5 × 5 × 16
```

Convolution:

```
5 × 5 filter
```

produces:

```
1 × 1 output
```

Since there are 400 filters:

```
1 × 1 × 400
```

output volume.

Visualization:

```
5×5×16
    ↓
Conv(400 filters, 5×5)
    ↓
1×1×400
```

Why Is This Equivalent?

Each FC neuron computes:

```
Weighted Sum
+
Bias
```

over all 400 inputs.

A:

```
5×5×16 filter
```

does exactly the same thing.

Therefore:

```
FC Layer
=
5×5 Convolution
```

mathematically.

Step 3: Convert the Second Fully Connected Layer

Original:

```
400
  ↓
400
```

FC layer.

Equivalent:

```
1×1×400
    ↓
1×1 Conv
(400 filters)
    ↓
1×1×400
```

Why 1×1 Works

Each filter:

```
1 × 1 × 400
```

looks across all 400 channels.

Thus it computes:

```
Linear Combination
of all 400 activations
```

which is exactly what an FC neuron does.

Step 4: Convert the Output Layer

Original:

```
400
 ↓
4
```

Equivalent:

```
1×1×400
    ↓
1×1 Conv
(4 filters)
    ↓
1×1×4
```

followed by:

```
Softmax
```

Final Fully Convolutional Network

The network becomes:

```
14×14×3
    ↓
5×5 Conv (16)
    ↓
10×10×16
    ↓
Max Pool
    ↓
5×5×16
    ↓
5×5 Conv (400)
    ↓
1×1×400
    ↓
1×1 Conv (400)
    ↓
1×1×400
    ↓
1×1 Conv (4)
    ↓
1×1×4
```

No fully connected layers remain.

Everything is now convolutional.

Why Do This?

At first glance, this seems pointless because:

```
FC Layer
=
Equivalent Conv Layer
```

and both produce the same output.

The important advantage appears in the next step.

Key Insight

A fully connected network requires:

```
One forward pass
for each cropped window
```

during sliding-window detection.

After converting everything to convolutions:

```
One forward pass
can evaluate many windows simultaneously
```

because convolutions naturally reuse computations across overlapping image regions.

Computational Benefit

Naive Sliding Window:

```
Window 1 → ConvNet
Window 2 → ConvNet
Window 3 → ConvNet
...
```

Thousands of independent forward passes.

Convolutional Sliding Window:

```
Entire Image
      ↓
Single ConvNet Pass
      ↓
Predictions for Many Locations
```

Shared computations dramatically reduce runtime.

Why This Matters Historically

This idea was one of the key breakthroughs that made ConvNet-based object detection practical.

Without it:

```
Sliding Windows
+
Deep ConvNet
```

would be prohibitively expensive.

With convolutional implementation:

```
Sliding Windows
+
Shared Convolution Computation
```

becomes much faster and forms the foundation of modern detection architectures.

Key Takeaways

Converting FC Layers

Fully Connected Layer	Equivalent Convolution
FC(400) from 5×5×16	400 filters of size 5×5×16
FC(400)	1×1 Conv with 400 filters

Fully Connected Layer	Equivalent Convolution
FC(4)	1×1 Conv with 4 filters

Important Concept

```
Fully Connected
=
Special Case of Convolution
```

when the filter covers the entire input volume.

Main Benefit

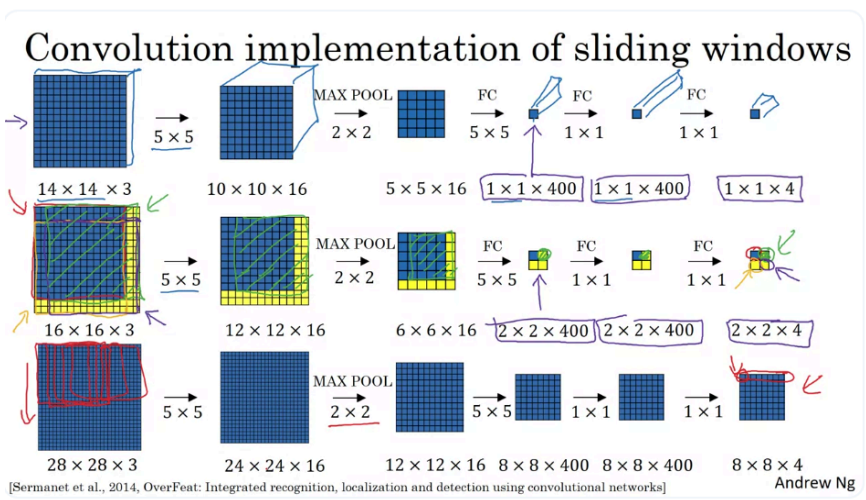
After converting FC layers to convolutional layers:

```
Entire Image
      ↓
Single Forward Pass
      ↓
Predictions at Multiple Locations
```

instead of running the ConvNet separately on thousands of cropped windows.

Preview

The next lecture uses this fully convolutional network to show how a single forward pass can perform the work of the entire sliding-window algorithm, making object detection dramatically faster.



Note that: The FC in the slide is the “FC layer has been turned into convolutional layers”

Convolutional Implementation of Sliding Windows

Motivation

In the previous lecture, we converted a standard ConvNet into a **fully convolutional network** by replacing fully connected layers with convolutional layers.

The goal of this lecture is to show how this conversion allows us to perform **Sliding Window Detection much more efficiently**.

The key idea is:

> Instead of running the ConvNet separately on every window, run the ConvNet once on the entire image and obtain predictions for many windows simultaneously.

This idea was popularized in the **OverFeat** paper by Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Robert Fergus, and Yann LeCun.

Review: Original Detector

Suppose our detector was trained on:

```
14 × 14 × 3 images
```

and outputs:

```
1 × 1 × 4
```

representing probabilities for:

```
Pedestrian
Car
Motorcycle
Background
```

Architecture:

```
14×14×3
    ↓
Conv
    ↓
10×10×16
    ↓
Max Pool
    ↓
5×5×16
    ↓
Conv (replacing FC)
    ↓
1×1×400
    ↓
1×1×400
    ↓
1×1×4
```

Naive Sliding Windows

Input Image

Suppose the test image is:

```
16 × 16 × 3
```

instead of:

```
14 × 14 × 3
```

The extra pixels create multiple possible 14×14 windows.

Four Possible Windows

Using a stride of 2:

```
+---------+---------+
| Window1 | Window2 |
|         |         |
+---------+---------+
| Window3 | Window4 |
|         |         |
+---------+---------+
```

The traditional sliding window algorithm would:

1. Crop Window 1

2. Run ConvNet
3. Crop Window 2
4. Run ConvNet
5. Crop Window 3
6. Run ConvNet
7. Crop Window 4
8. Run ConvNet

Total:

```
4 independent forward passes
```

Problem: Massive Computation Duplication

Notice:

```
Window 1
Window 2
```

overlap heavily.

Similarly:

```
Window 1
Window 3
```

share most pixels.

Yet the ConvNet recomputes all intermediate activations repeatedly.

This causes enormous redundancy.

Convolutional Solution

Instead of cropping four windows and running the network four times:

```
Crop → ConvNet
Crop → ConvNet
Crop → ConvNet
Crop → ConvNet
```

feed the entire:

```
16×16×3
```

image through the fully convolutional network once.

Layer-by-Layer Dimensions

First Convolution

Original:

```
14×14×3
    ↓
10×10×16
```

Now:

```
16×16×3
    ↓
12×12×16
```

because the same filters are applied to a larger image.

Max Pooling

```
12×12×16
    ↓
6×6×16
```

5×5 Convolution (Former FC Layer)

Previously:

```
5×5×16
    ↓
1×1×400
```

Now:

```
6×6×16
    ↓
2×2×400
```

Instead of one output location, we obtain four.

1×1 Convolution

```
2×2×400
    ↓
2×2×400
```

Final 1×1 Convolution

```
2×2×400
    ↓
2×2×4
```

Final output:

```
2×2×4
```

instead of:

```
1×1×4
```

Interpretation of the 2×2×4 Output

Each spatial location corresponds to one sliding window prediction.

```
+-----------+-----------+
| Output11  | Output12  |
+-----------+-----------+
| Output21  | Output22  |
+-----------+-----------+
```

Specifically:

Output Cell	Corresponding Window
Top Left	Upper-left 14×14 patch
Top Right	Upper-right 14×14 patch
Bottom Left	Lower-left 14×14 patch
Bottom Right	Lower-right 14×14 patch

Each output contains:

```
[Pedestrian, Car, Motorcycle, Background]
```

probabilities.

Why Does This Work?

Consider the upper-right window.

If we had:

```
Crop Window
    ↓
Run ConvNet
```

the intermediate activations would be exactly the same as the activations generated by the corresponding region of the larger convolutional computation.

Because convolutions are local operations, the network naturally computes all overlapping windows simultaneously.

Main Advantage

Naive Approach

```
Window 1 → Full ConvNet
Window 2 → Full ConvNet
Window 3 → Full ConvNet
Window 4 → Full ConvNet
```

Many duplicated computations.

Convolutional Approach

```
Entire Image
      ↓
Single Forward Pass
      ↓
Predictions for All Windows
```

Shared intermediate computations eliminate redundancy.

Larger Example: 28×28 Image

Suppose we input:

```
28 × 28 × 3
```

instead of:

```
16 × 16 × 3
```

Running the same fully convolutional network now produces:

```
8 × 8 × 4
```

output volume.

Meaning of the 8×8 Output Grid

```
8 × 8
```

means:

```
64 window positions
```

were evaluated simultaneously.

Each output cell corresponds to a different:

```
14 × 14 window
```

in the original image.

Visualization

```
Output Grid

o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
```

Each `o` represents one object-classification prediction.

Effective Stride

The lecture notes that:

```
Max Pooling = 2×2
```

causes the detector to move with an effective stride of:

```
2 pixels
```

in the original image.

Thus the output grid naturally corresponds to sliding windows spaced every 2 pixels.

Why This Was Important

Before this idea:

```
Thousands of windows
      ↓
Thousands of ConvNet evaluations
```

After this idea:

```
One ConvNet evaluation
      ↓
Thousands of window predictions
```

This dramatically reduced computational cost and made ConvNet-based object detection practical.

Relationship to Sliding Windows

The algorithm is still fundamentally:

```
Sliding Windows Detection
```

but implemented more efficiently.

Conceptually:

```
Crop Every Window
```

becomes:

```
Run Convolution Once
```

The detector still evaluates many windows, but now all windows share computation.

Key Takeaways

Core Idea

Convert the classifier into a fully convolutional network and run it on the entire image.

Example

```
Input:
16×16×3

Output:
2×2×4
```

Each output cell corresponds to one 14×14 sliding window.

Larger Example

```
Input:
28×28×3

Output:
8×8×4
```

Each of the 64 cells is a detector prediction.

Major Benefit

Instead of:

```
Many Crops
+
Many Forward Passes
```

we obtain:

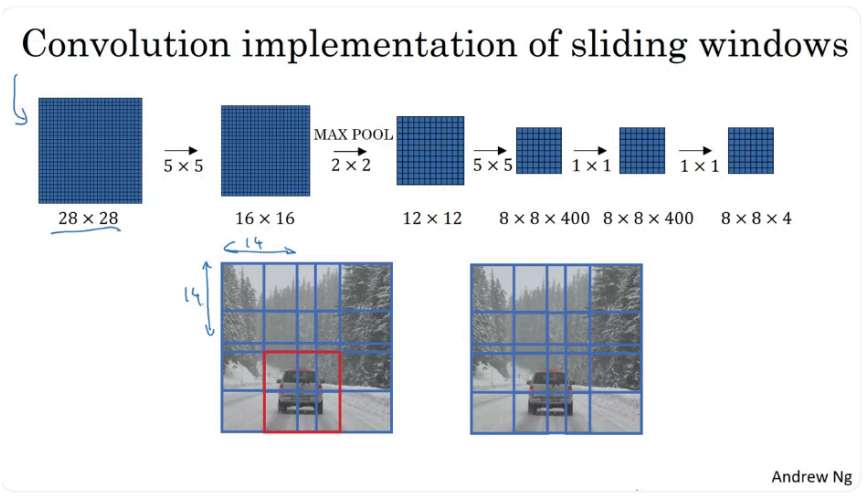
```
One Forward Pass
+
Shared Computation
```

which is dramatically faster.

Limitation

Although much faster than naive sliding windows, the detector still evaluates a fixed grid of locations and scales.

This limitation motivates the next generation of object detectors, including region proposal methods and YOLO-style detectors.



Recap: Convolutional Sliding Windows

Traditional Sliding Windows

In the original sliding windows algorithm, object detection is performed by repeatedly cropping regions from the image and running the ConvNet on each crop independently.

Process:

```
Image
  ↓
Crop Window #1 (14×14)
  ↓
ConvNet
  ↓
Prediction

Crop Window #2 (14×14)
  ↓
ConvNet
  ↓
Prediction

Crop Window #3 (14×14)
  ↓
ConvNet
  ↓
Prediction

...
```

The detector keeps sliding the window across the image until every possible region has been evaluated.

Eventually, one of the windows may contain the object (e.g., a car), causing the classifier to output a high confidence score.

Drawback

This approach is computationally expensive because:

- Each window requires a separate forward pass.
- Neighboring windows overlap heavily.
- The same computations are repeated many times.

Convolutional Sliding Windows

Instead of processing windows one by one:

```
Window 1 → ConvNet
Window 2 → ConvNet
Window 3 → ConvNet
...
```

we process the entire image at once.

Example:

```
28×28 Image
      ↓
Fully Convolutional Network
      ↓
8×8×4 Output
```

Each output cell corresponds to one sliding-window prediction.

Visualization:

```
Input Image
      ↓
Single Forward Pass
      ↓

o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
o o o o o o o o
```

where each `o` is equivalent to running the classifier on a different image window.

Key Advantage

Before

```
Many Windows
      ↓
Many ConvNet Evaluations
```

After

```
Entire Image
      ↓
One ConvNet Evaluation
```

```
      ↓
Predictions for All Windows
```

The network reuses computations across overlapping regions, making detection dramatically faster.

Why It Works

A fully convolutional network naturally preserves spatial information.

When a larger image is passed through the network:

```
Larger Input
      ↓
Larger Feature Maps
      ↓
Grid of Predictions
```

Each prediction corresponds to a different receptive field (window) in the original image.

Thus:

```
Many Sliding Windows
```

becomes

```
One Convolutional Forward Pass
```

while producing essentially the same results.

Remaining Limitation

Although convolutional sliding windows is much faster, it still has an important weakness:

Poor Bounding Box Precision

The detector predicts objects only at predefined grid locations.

Example:

```
+-----------+
|   Car     |
+-----------+
```

The object may lie:

- Between two grid cells.
- Between two window positions.
- Between two scales.

As a result:

- Object classification may be correct.
- Bounding box localization may be inaccurate.

The detector can identify that:

> "There is a car somewhere around here."

but cannot precisely determine:

> "The car occupies exactly this bounding box."

Key Takeaways

Traditional Sliding Windows

```
Crop Window
     ↓
Run ConvNet
     ↓
Repeat Thousands of Times
```

- Simple
- Very slow

Convolutional Sliding Windows

```
Entire Image
      ↓
Fully Convolutional Network
      ↓
Grid of Predictions
```

- Same underlying idea
- Much faster
- Shares computations across overlapping windows

Remaining Problem

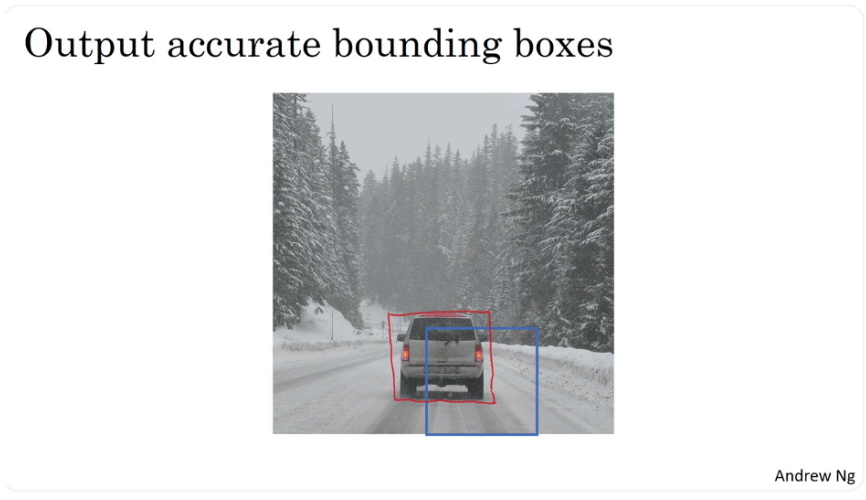
```
Fast Detection
      ✓

Accurate Localization
      ✗
```

Bounding boxes are still tied to a coarse grid, limiting localization accuracy.

This motivates the next improvement: learning more precise bounding box predictions instead of relying solely on fixed sliding-window positions.

YOLO Algorithm - Bounding Box Predictions

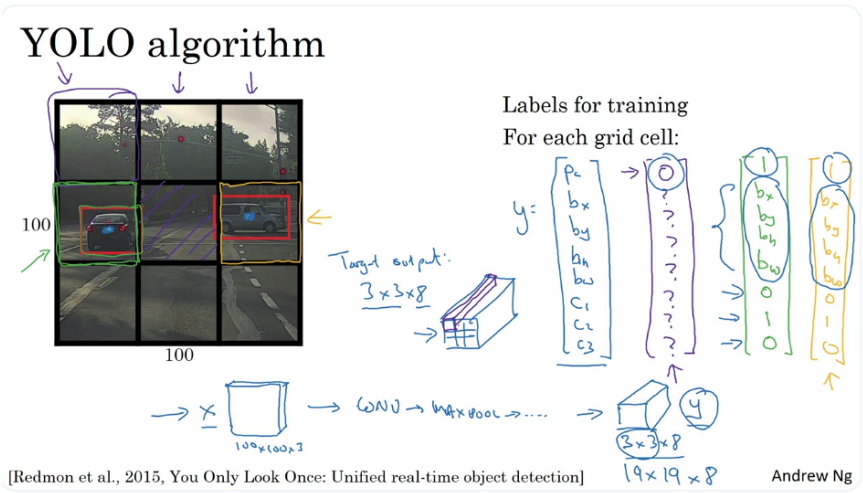


In the last video, you learned how to use a convolutional implementation of sliding windows, and that's more computationally efficient, but it still has a problem of not quite outputting the most accurate bounding boxes.

In this video, let's see how you can get your bounding box predictions to be more accurate.

With sliding windows, you take this, um, discrete set of locations and run the classifier through it. And in this case, none of the boxes really match up perfectly with the position of the car. So maybe that box (blue square in the image) was the best match.

And also, it looks like in the ground truth, the perfect bounding box isn't even quite square. It's actually, you know, has a slightly wider, um, rectangle, a slightly horizontal aspect ratio.
So is there a way to get this algorithm to output more accurate bounding boxes?



YOLO: You Only Look Once

Motivation

The convolutional sliding windows approach solved the **computational cost problem**, but it still has a major weakness:

> Bounding boxes are constrained by the sliding-window grid.

Example:

```
+-------------+
|     Car     |
+-------------+
```

The object may lie between window locations.

As a result:

- The classifier may detect the car.
- The predicted bounding box may be poorly aligned.
- Bounding box shapes are limited by predefined window sizes.

YOLO addresses this issue by predicting bounding boxes **directly** instead of relying on fixed sliding windows.

Core Idea of YOLO

YOLO stands for:

```
You Only Look Once
```

Instead of:

```
Image
 ↓
Many Sliding Windows
 ↓
Many Predictions
```

YOLO performs:

```
Image
 ↓
Single Neural Network
```

```
 ↓
All Object Predictions
```

in one forward pass.

Step 1: Divide the Image into a Grid

Suppose the input image is:

```
100 × 100
```

For illustration, divide it into:

```
3 × 3 grid
```

```
+-----+-----+-----+
| 1   | 2   | 3   |
+-----+-----+-----+
| 4   | 5   | 6   |
+-----+-----+-----+
| 7   | 8   | 9   |
+-----+-----+-----+
```

In practice:

```
19 × 19
```

or larger grids are commonly used.

Step 2: Assign Objects to Grid Cells

YOLO assigns each object according to:

> The grid cell containing the object's center point (midpoint).

Example:

```
+-----+-----+-----+
|     |     |     |
+-----+-----+-----+
| Car |     | Car |
+-----+-----+-----+
|     |     |     |
+-----+-----+-----+
```

Even if an object spans multiple cells:

```
+-----+-----+-----+
|     |     |     |
+-----+-----+-----+
|XXXXX|XXXXX|     |
|XXXXX|XXXXX|     |
+-----+-----+-----+
```

it is assigned only to the cell containing its center.

Output Vector for Each Grid Cell

For each grid cell, YOLO predicts:

```
[
pc,
bx,
by,
bh,
bw,
c1,
c2,
c3
]
```

Total:

```
8 values
```

Object Presence

```
pc
```

Value	Meaning
1	Object exists
0	No object

Bounding Box

```
bx
by
bh
bw
```

represent:

- Center x-coordinate
- Center y-coordinate
- Height
- Width

Class Labels

Assuming three object classes:

```
c1 = pedestrian
c2 = car
c3 = motorcycle
```

Example:

```
[1,bx,by,bh,bw,0,1,0]
```

means:

```
Object exists
Class = Car
```

Example Labels

Empty Grid Cell

```
[
0,
?,
?,
?,
?,
?,
?,
?,
?
]
```

where:

```
?
```

means "don't care".

Grid Cell Containing a Car

```
[
1,
bx,
by,
bh,
bw,
0,
1,
0
]
```

Output Volume

Since:

```
3 × 3 grid
```

contains:

```
9 cells
```

and each cell outputs:

```
8 values
```

the target output becomes:

```
3 × 3 × 8
```

Visualization

```
Grid Cell
    ↓
1×1×8 Vector
```

For all cells:

```
3×3 Grid
    ↓
3×3×8 Tensor
```

Training the Network

Input:

```
100 × 100 × 3
```

Output:

```
3 × 3 × 8
```

Training process:

```
Image
   ↓
ConvNet
   ↓
3×3×8 Output
```

The network is trained using backpropagation to predict the target tensor.

Prediction at Test Time

At inference:

```
Input Image
      ↓
ConvNet
      ↓
3×3×8 Output
```

For each grid cell:

1. Check `pc`
2. Determine object class
3. Read bounding box coordinates

Example:

```
Cell (2,1)
pc = 1
Class = Car
Bounding Box = (...)
```

Why YOLO Produces Better Bounding Boxes

Sliding Windows

Bounding boxes are tied to:

```
Window Locations
```

and

```
Window Sizes
```

Example:

```
□
□
□
```

The detector can only choose among predefined windows.

YOLO

The network predicts:

```
bx
by
bh
bw
```

directly.

Therefore it can output:

- Any center location
- Any width
- Any height
- Any aspect ratio

Example:

```
Wide rectangle
Tall rectangle
Square
```

all can be predicted directly.

Why YOLO Is Fast

A naive implementation might require:

```
361 grid cells
```

for a:

```
19 × 19
```

grid.

YOLO does **not** run 361 independent detectors.

Instead:

```
One Image
     ↓
One ConvNet
     ↓
All 361 Predictions
```

This is a fully convolutional computation with extensive parameter sharing.

Limitation

YOLO assumes:

> At most one object center per grid cell.

If two objects have centers inside the same grid cell:

```
+-------+
| Car   |
| Person|
+-------+
```

the original YOLO formulation struggles.

The course notes that this issue will be addressed later.

Comparison: Sliding Windows vs YOLO

Feature	Sliding Windows	YOLO
Detection Method	Scan windows	Predict directly
Number of Forward Passes	Many	One
Speed	Slow	Fast
Bounding Box Accuracy	Limited by windows	Direct prediction
Real-Time Detection	Difficult	Yes

Key Takeaways

YOLO Core Idea

```
Image
  ↓
Single ConvNet
  ↓
Grid of Predictions
```

Grid Assignment Rule

```
Object
   ↓
Assign to cell containing object's center
```

Per-Cell Output

```
[
pc,
bx,
by,
bh,
bw,
c1,
c2,
c3
]
```

Output Tensor

For a 3×3 grid:

```
3 × 3 × 8
```

For a 19×19 grid:

```
19 × 19 × 8
```

Major Advantages

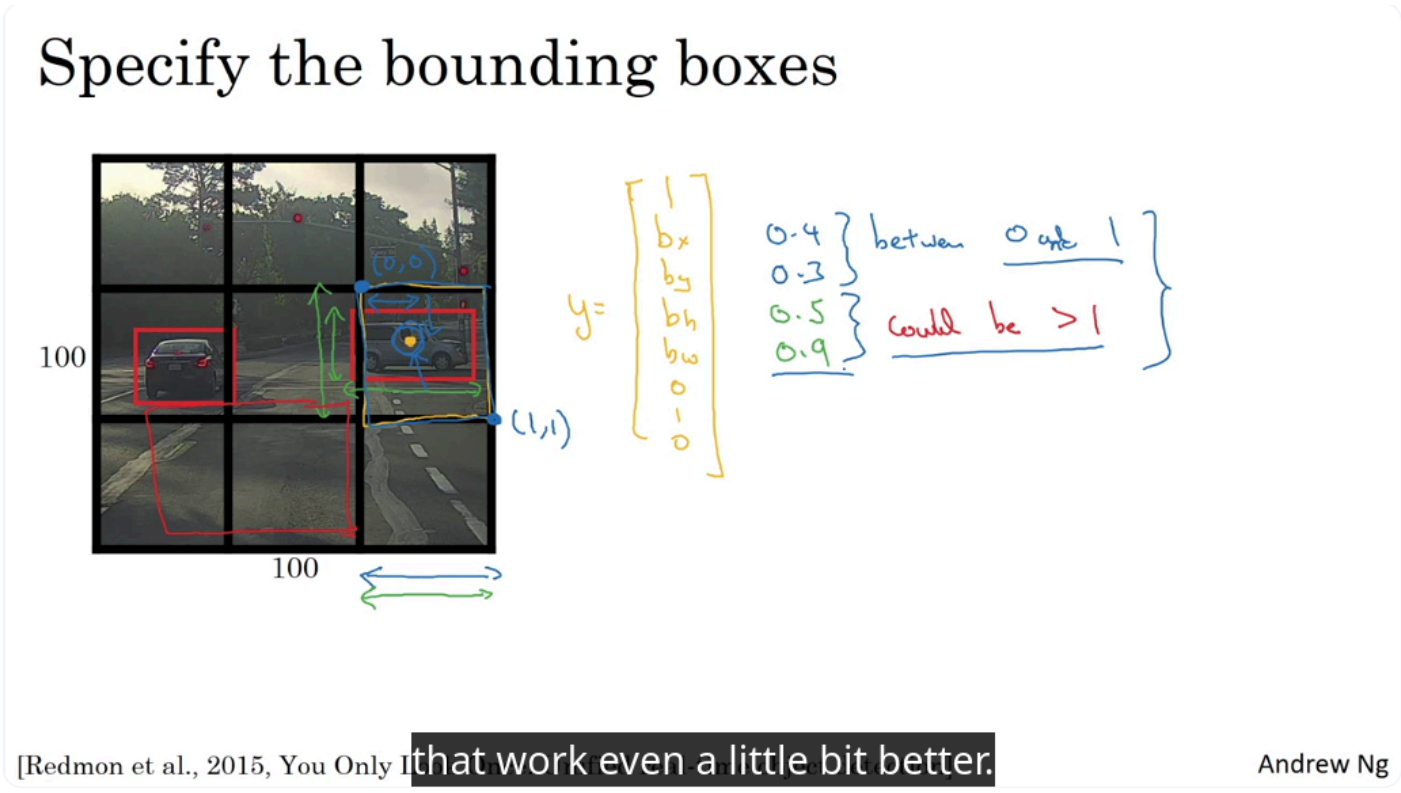
1. Direct bounding box prediction.
2. Arbitrary aspect ratios.
3. More accurate localization.
4. Fully convolutional.
5. Extremely fast.
6. Suitable for real-time object detection.

YOLO can be viewed as combining the ideas of:

- Classification
- Localization
- Convolutional computation

into a single end-to-end trainable object detection framework.

YOLO Bounding Box Parameterization



[Redmon et al., 2015, You Only L

Target Label Format

For a grid cell containing an object:

```
y = [Pc, Bx, By, Bh, Bw, C1, C2, C3]

Example:
[1, 0.4, 0.3, 0.5, 0.9, 0, 1, 0]
```

- `Pc = 1`: object exists in the grid cell
- `Bx, By`: center coordinates of the bounding box
- `Bh, Bw`: height and width of the bounding box

- `C1, C2, C3`: class labels

Bounding Box Coordinates

YOLO defines the grid cell as a normalized coordinate system:

- Top-left corner: `(0, 0)`
- Bottom-right corner: `(1, 1)`

The bounding box center is represented relative to the assigned grid cell.

Example:

- `Bx ≈ 0.4`
- `By ≈ 0.3`

Bounding Box Size

Bounding box dimensions are expressed as fractions relative to the grid cell size.

Example:

- `Bh ≈ 0.5 (could be greater than 1)`
- `Bw ≈ 0.9`

Parameter Constraints

Center coordinates

- `Bx ∈ [0, 1]`
- `By ∈ [0, 1]`

Reason:

- The object center must lie inside its assigned grid cell.
- Otherwise, the object would belong to a different cell.

Bounding box dimensions

- `Bh ≥ 0`
- `Bw ≥ 0`

These values can be **greater than 1** when the bounding box extends beyond the grid cell.

Alternative Parameterizations

The presented representation is a simple and reasonable convention.

YOLO papers introduce more advanced parameterizations, such as:

- Sigmoid functions to constrain `Bx` and `By` to `[0, 1]`
- Exponential parameterization to ensure `Bh` and `Bw` remain non-negative

These approaches generally achieve better performance.

Notes on the YOLO Paper

- The YOLO research paper is considered difficult to read.
- Even experienced researchers may struggle to understand all implementation details.
- It is common to:
  - Read open-source implementations
  - Contact the authors
  - Consult other researchers

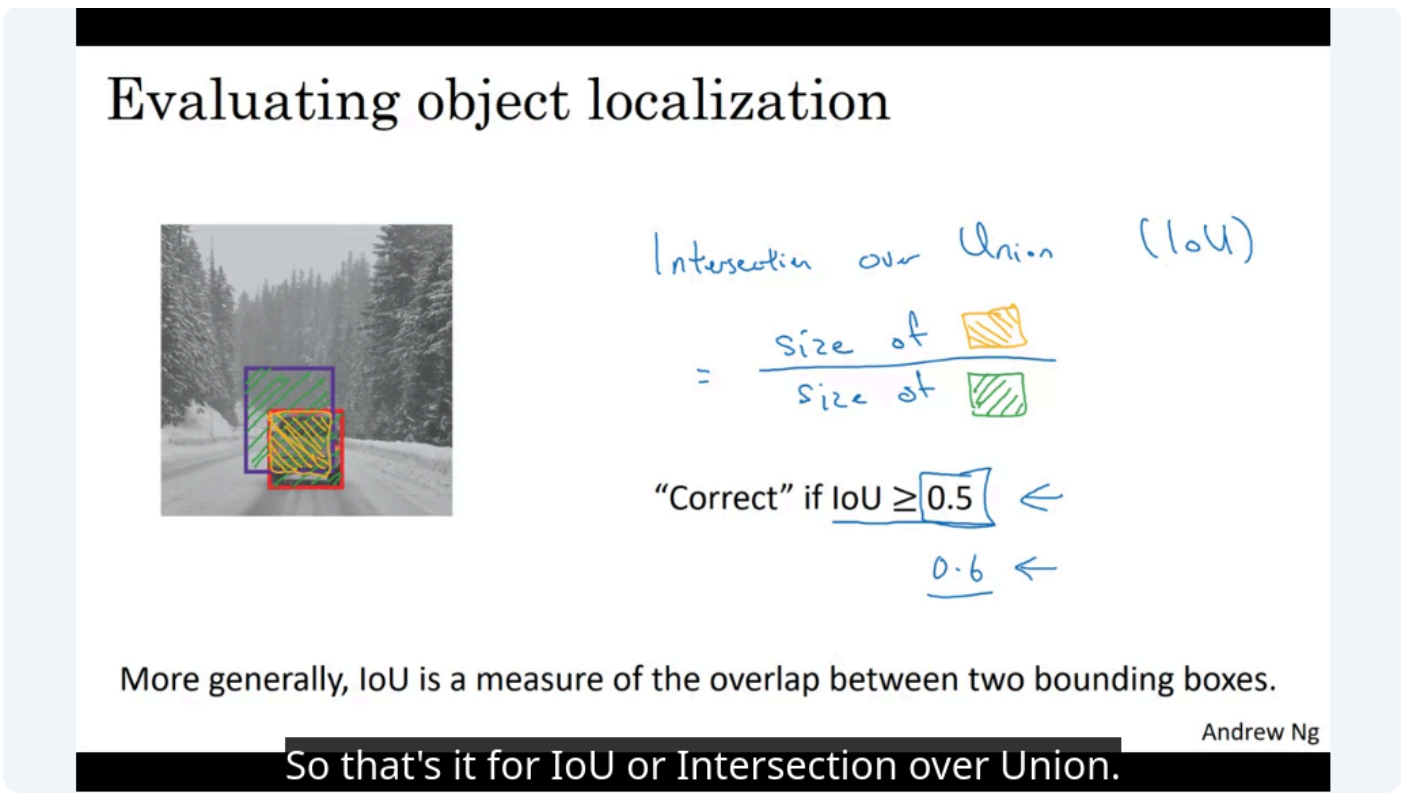
Having difficulty understanding the paper is normal.

Key Takeaway

You should now understand the basic YOLO representation:

- Objects are assigned to grid cells.
- Bounding box centers are normalized within the grid cell.
- Bounding box sizes are represented relative to the grid cell and may exceed 1.
- More advanced parameterizations exist but follow the same fundamental idea.

Intersection Over Union (IoU)



Intersection over Union (IoU)

Purpose

Intersection over Union (IoU) is a metric used to:

- Evaluate object detection performance
- Measure how well a predicted bounding box matches the ground-truth bounding box

It is also used in techniques such as **Non-Max Suppression (NMS)**.

Definition

IoU is defined as:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

where:

- **Intersection**: area shared by both bounding boxes
- **Union**: total area covered by either bounding box

IoU Values

- **IoU = 1**
  - Predicted box and ground-truth box overlap perfectly.
- **0 < IoU < 1**
  - Partial overlap between the two boxes.
- **IoU = 0**

- No overlap.

Evaluation Threshold

A common convention is:

- **IoU ≥ 0.5 → Correct detection**
- **IoU < 0.5 → Incorrect detection**

This threshold is widely used but is not based on a theoretical requirement.

Choosing the Threshold

Higher thresholds require more accurate localization.

Common choices:

IoU Threshold	Interpretation
0.5	Standard convention
0.6	More stringent
0.7	Even stricter

Thresholds below **0.5** are rarely used.

IoU as a Similarity Measure

IoU measures the similarity between two bounding boxes by comparing:

- their overlapping area (intersection)
- their combined area (union)

A larger IoU indicates that the two boxes are more similar.

Applications

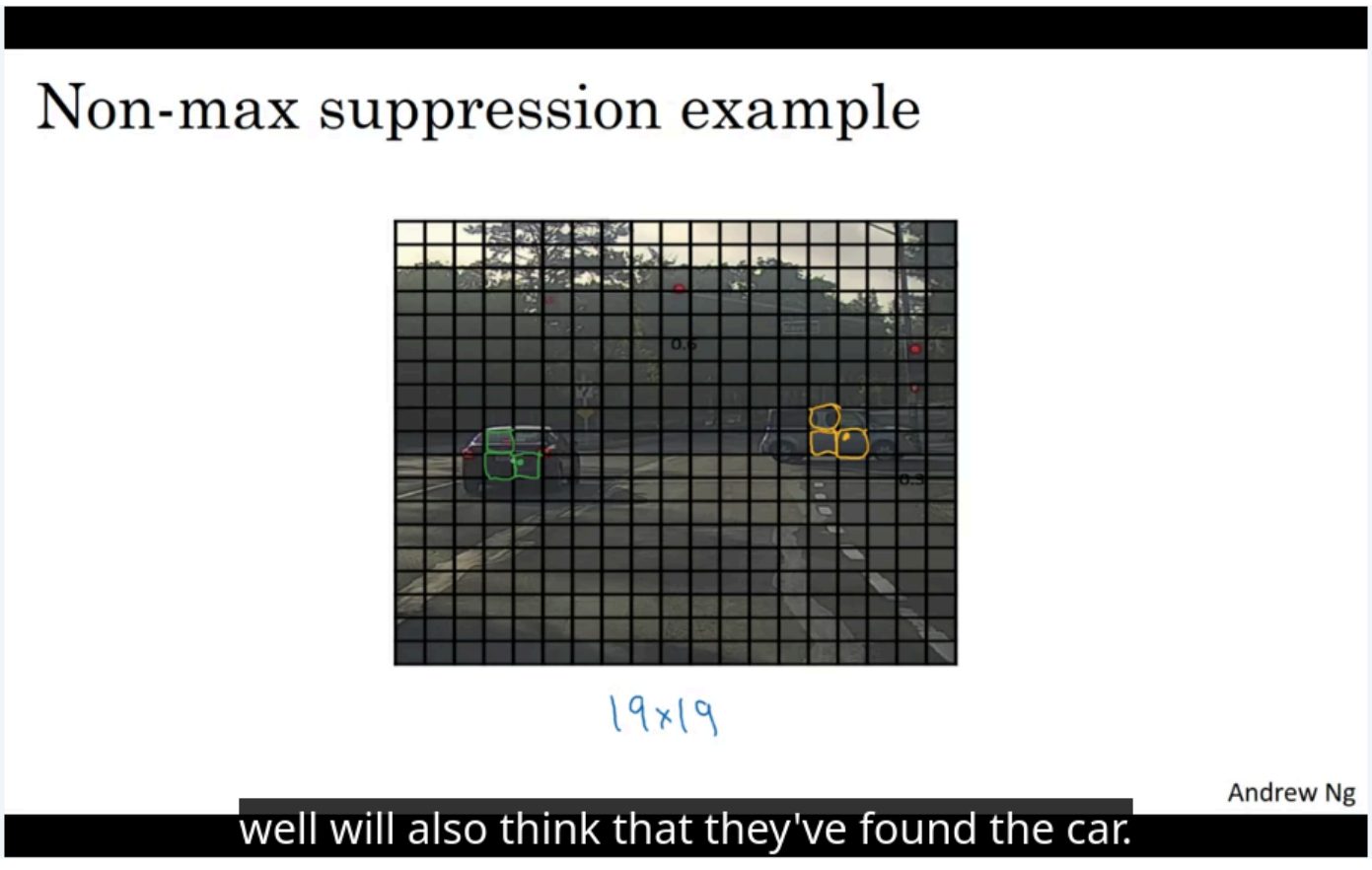
IoU is used for:

- Evaluating object localization accuracy
- Determining whether a detected object is correctly localized
- Measuring overlap between bounding boxes
- Supporting algorithms such as **Non-Max Suppression (NMS)**

Key Takeaways

- IoU quantifies the overlap between predicted and ground-truth bounding boxes.
- It is computed as **intersection area divided by union area**.
- **IoU ≥ 0.5** is the most common criterion for a correct detection.
- Higher IoU values indicate more accurate object localization.
- IoU is both an evaluation metric and a fundamental component of object detection algorithms.

Non-max Suppression



Problem

In object detection, the algorithm may detect the same object multiple times.

Instead of producing one bounding box for one object, several grid cells may predict that they found the same object.

Example

Suppose the model needs to detect:

- Pedestrians
- Cars
- Motorcycles

The image is divided into a **19 by 19 grid**.

Technically, each object has only one midpoint, so each object should be assigned to only one grid cell.

Why Multiple Detections Happen

In practice, the model runs object classification and localization for every grid cell.

Because of this, several nearby grid cells may predict the same object.

For example:

- One grid cell may correctly detect a car.
- Nearby grid cells may also think they found the same car.
- This can happen for multiple objects in the image.

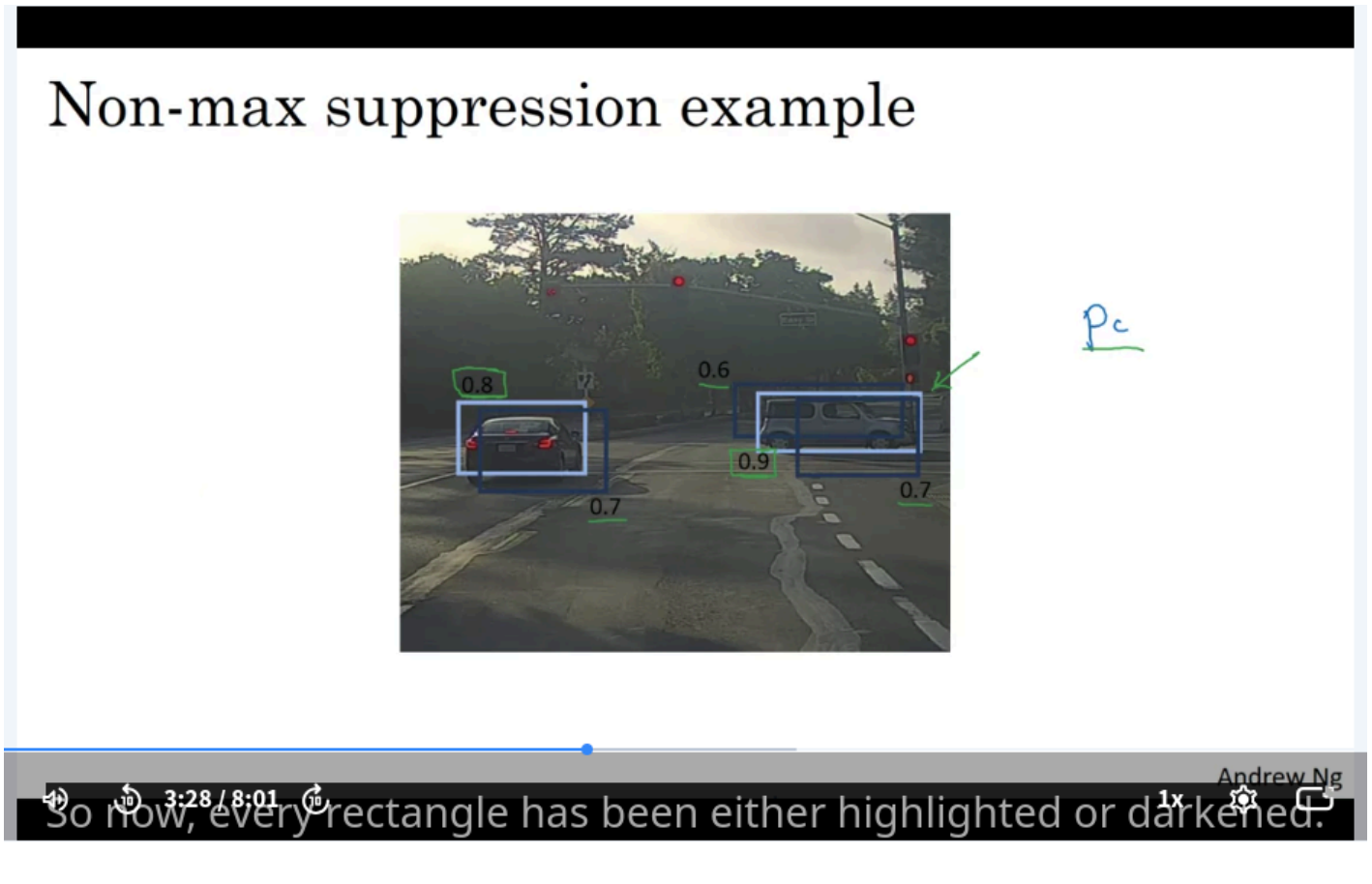
Purpose of Non-Max Suppression

Non-Max Suppression is used to make sure that each object is detected only once.

It removes duplicate detections and keeps the most confident bounding box for each object.

Key Takeaway

Non-Max Suppression helps object detection algorithms avoid multiple detections of the same object.



Motivation

Since object classification and localization are performed on every grid cell, many grid cells may predict the same object with high confidence.

As a result, the model can produce multiple bounding boxes for a single object.

Goal

Non-Max Suppression (NMS) removes duplicate detections and keeps only one bounding box for each object.

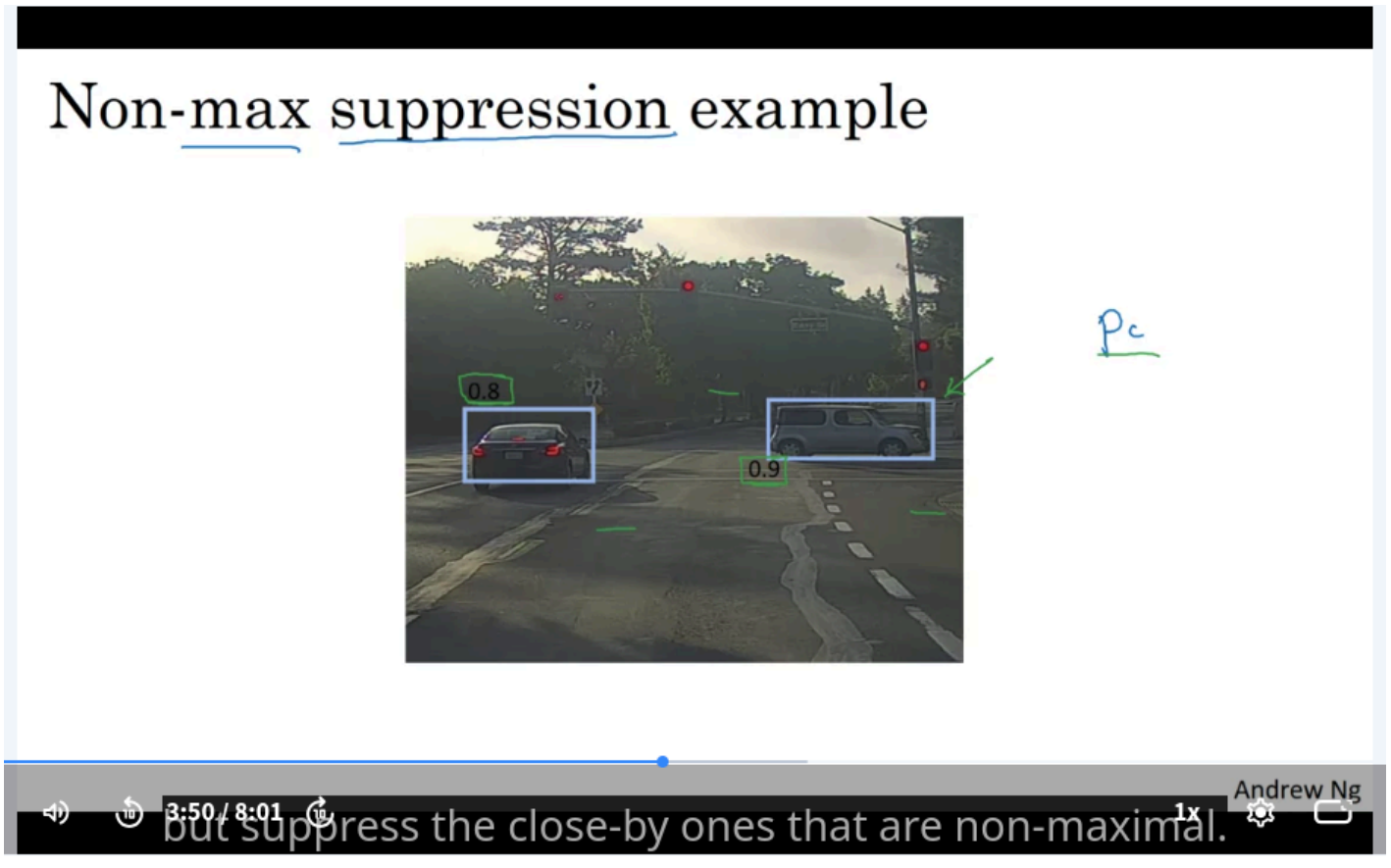
Algorithm Steps

1. Collect All Detections

Each predicted bounding box has:

- Bounding box coordinates
- Detection confidence (`Pc`)
  - In practice, the score is often `Pc × C1`, `Pc × C2`, or `Pc × C3` depending on the object class.

2. Select the Highest-Confidence Detection



Find the bounding box with the largest confidence score.

Example:

- `Pc = 0.9`

This box is accepted as a final detection.

3. Suppress Overlapping Boxes

Compare the selected box with all remaining boxes.

If another box has a **high IoU** with the selected box, it is removed (suppressed).

Example:

- Boxes with confidence `0.7` and `0.6` have high overlap with the `0.9` box.
- These boxes are suppressed.

4. Repeat

From the remaining unsuppressed boxes:

- Select the highest-confidence box.
- Keep it as a final detection.
- Suppress all boxes that highly overlap with it.

Continue until every bounding box has been either:

- Selected as a final prediction
- Suppressed

Final Output

After removing all suppressed boxes, only the selected bounding boxes remain.

Each object is represented by a single detection.

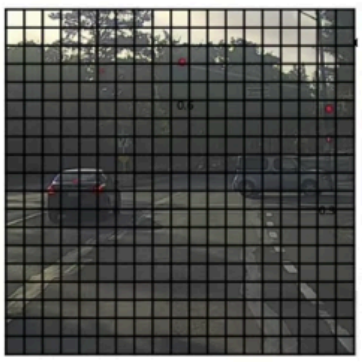
Meaning of "Non-Max Suppression"

- **Max**: keep the bounding box with the highest confidence score.
- **Non-Max**: all nearby boxes with lower confidence.
- **Suppression**: remove those lower-confidence overlapping boxes.

Key Takeaways

- NMS eliminates duplicate detections of the same object.
- It always keeps the highest-confidence prediction first.
- Bounding boxes with high IoU relative to the selected box are suppressed.
- The final result contains one bounding box per detected object.

Non-max suppression algorithm



19× 19

Each output prediction is: p_c, b_x, b_y, b_h, b_w

Discard all boxes with $p_c \leq 0.6$

While there are any remaining boxes:

- Pick the box with the largest p_c Output that as a prediction.
- Discard any remaining box with IoU ≥ 0.5 with the box output in the previous step

Andrew Ng

with the box that you just output in the previous step.

Non-Max Suppression Algorithm Details

Output Representation

For a **19 × 19 grid**, the model produces a:

```
19 × 19 × 8
```

output volume.

For simplicity, when detecting only **one object class (car)**, each grid cell outputs:

```
[Pc, Bx, By, Bh, Bw]
```

where:

- `Pc`: probability that an object exists
- `Bx, By`: bounding box center
- `Bh, Bw`: bounding box size

(Class probabilities `C1, C2, C3` are omitted in this simplified example.)

Step 1: Remove Low-Confidence Predictions

Discard all bounding boxes with:

```
Pc ≤ threshold
```

Example:

```
threshold = 0.6
```

Only boxes with sufficiently high confidence are kept for further processing.

Step 2: Select the Highest-Confidence Box

Among all remaining boxes:

- Find the box with the highest `Pc`

- Output it as a final prediction

This box is considered the most reliable detection.

Step 3: Suppress Overlapping Boxes

For the selected box:

- Compute IoU with every remaining box.
- Remove boxes whose IoU exceeds a predefined threshold.

These boxes are treated as duplicate detections of the same object.

Step 4: Repeat

Repeat Steps 2 and 3 until every bounding box has been:

- Output as a final prediction, or
- Discarded because of low confidence or high overlap.

Algorithm Summary

```
1. Discard boxes with Pc ≤ threshold.
2. While boxes remain:
    a. Select box with highest Pc.
    b. Output it as a prediction.
    c. Remove remaining boxes with high IoU.
3. Return all selected boxes.
```

Multiple Object Classes

For multiple classes (e.g., pedestrians, cars, motorcycles):

The output vector includes additional class probabilities.

Instead of applying NMS once:

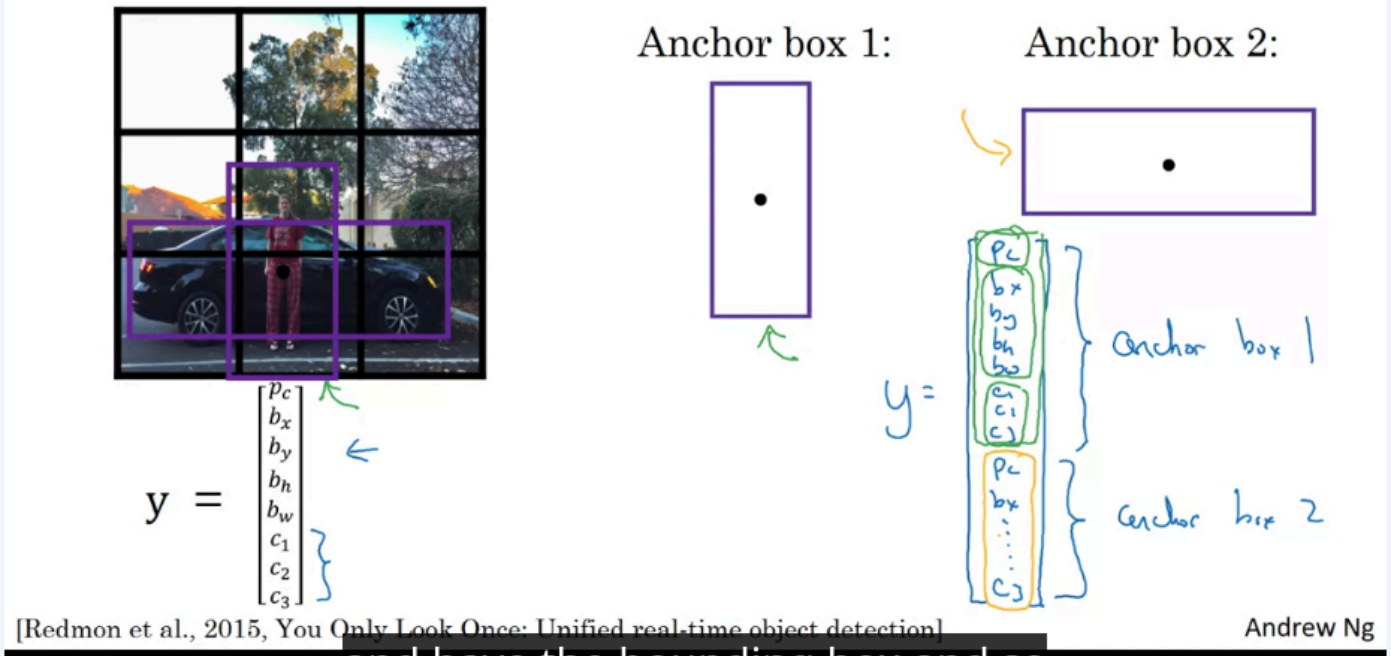
- Run Non-Max Suppression **independently for each object class**.

This prevents detections of different classes from suppressing each other.

Key Takeaways

- Start by filtering out low-confidence predictions.
- Always keep the highest-confidence bounding box.
- Suppress nearby boxes with high IoU.
- Repeat until no candidate boxes remain.
- For multi-class detection, apply NMS separately to each class.
- NMS is a key component that significantly improves YOLO detection quality before introducing **anchor boxes**.

Anchor Boxes



and have the bounding box and so

Motivation

In the basic YOLO approach, **each grid cell can detect only one object**.

This becomes a problem when multiple objects have their centers inside the same grid cell.

Example:

- A pedestrian and a car have nearly the same midpoint.
- Both belong to the same grid cell.
- A single prediction vector cannot represent both objects simultaneously.

Idea of Anchor Boxes

Anchor boxes allow **multiple predictions per grid cell**.

Instead of producing one prediction, each grid cell predicts one object for each predefined anchor box.

In practice:

- 2 anchor boxes can be used for simplicity.
- Real implementations often use **5 or more anchor boxes**.

Predefined Anchor Shapes

Before training, define several anchor box shapes.

Example:

- **Anchor Box 1:** tall and narrow
- **Anchor Box 2:** wide and short

Each object is assigned to the anchor box whose shape is most similar to its bounding box.

Output Representation

Without anchor boxes:

```
[Pc, Bx, By, Bh, Bw, C1, C2, C3]
```

With **2 anchor boxes**, the prediction vector becomes:

```
[Pc, Bx, By, Bh, Bw, C1, C2, C3, Pc, Bx, By, Bh, Bw, C1, C2, C3]
Anchor Box 1:
[Pc, Bx, By, Bh, Bw, C1, C2, C3]
```

```
Anchor Box 2:
[Pc, Bx, By, Bh, Bw, C1, C2, C3]
```

Each anchor box has its own independent prediction.

Object Assignment

Pedestrian

If the pedestrian's shape is closer to **Anchor Box 1**:

- `Pc = 1`
- Bounding box parameters (`Bx, By, Bh, Bw`) describe the pedestrian.
- Class prediction indicates **pedestrian**.

Car

If the car's shape is closer to **Anchor Box 2**:

- `Pc = 1`
- Bounding box parameters describe the car.
- Class prediction indicates **car**.

Thus, both objects can be represented within the same grid cell.

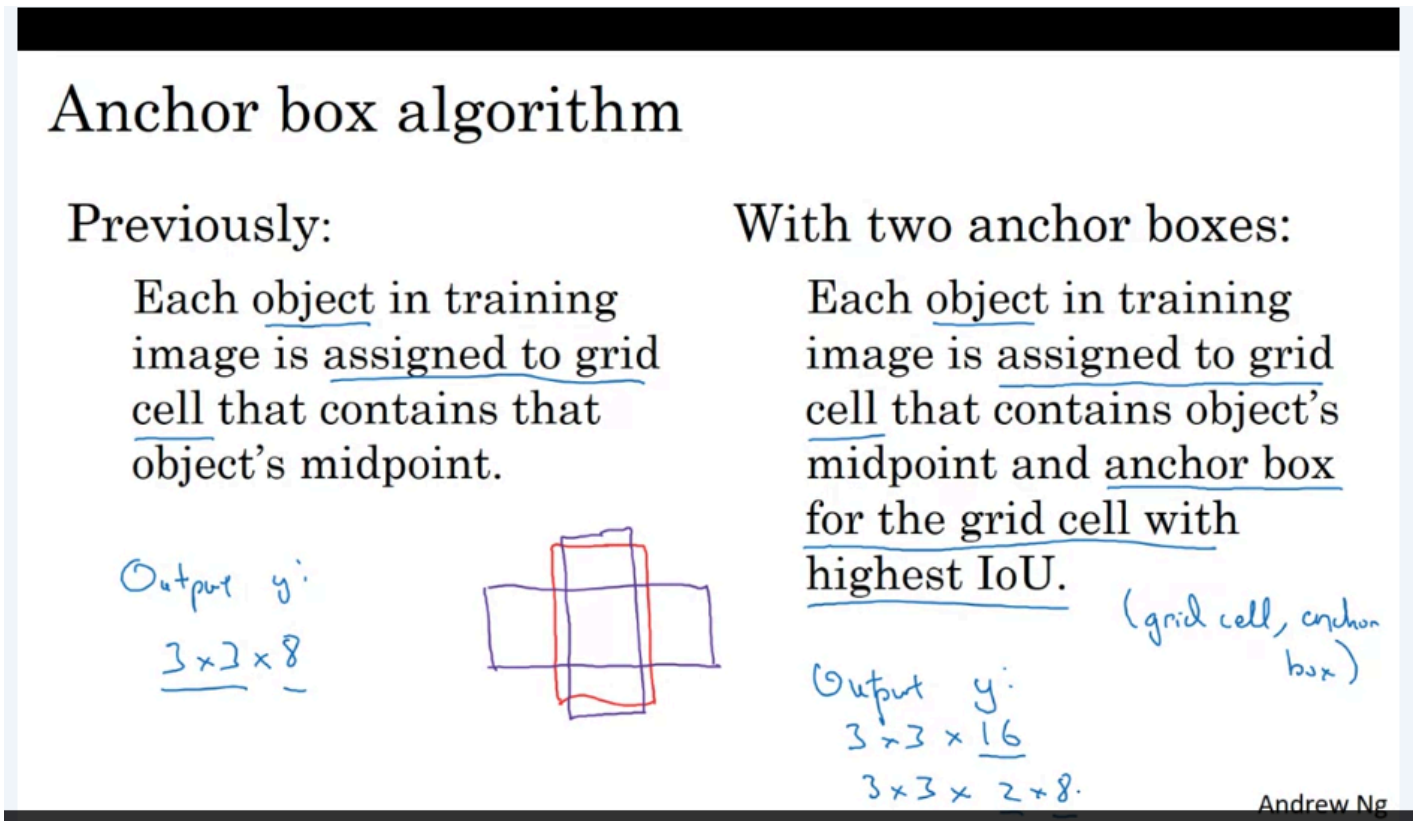
Benefits

Anchor boxes enable:

- Multiple object detections within a single grid cell.
- Better handling of objects with different aspect ratios.
- Independent predictions for different object shapes.

Key Takeaways

- A standard YOLO grid cell predicts only one object.
- Anchor boxes introduce multiple prediction slots per grid cell.
- Each anchor box corresponds to a predefined shape.
- Objects are assigned to the anchor box with the most similar shape.
- This allows multiple objects sharing the same grid cell to be detected simultaneously.



Training with Anchor Boxes

Before Using Anchor Boxes

Each object is assigned to:

- The grid cell containing its midpoint.

For a **3 × 3 grid**, the output tensor is:

```
3 × 3 × 8
```

Each grid cell predicts:

```
[Pc, Bx, By, Bh, Bw, C1, C2, C3]
```

where:

- `Pc`: object confidence
- `Bx, By`: bounding box center
- `Bh, Bw`: bounding box size
- `C1, C2, C3`: class probabilities

After Using Anchor Boxes

Each object is still assigned to:

- The grid cell containing its midpoint,

but it is also assigned to:

- the **anchor box with the highest IoU** relative to the object's shape.

Thus, an object is assigned to a:

```
(grid cell, anchor box)
```

pair.

Anchor Box Assignment

Given multiple predefined anchor boxes:

1. Compare the object's ground-truth bounding box with each anchor box.
2. Compute the IoU for each anchor box.
3. Select the anchor box with the highest IoU.
4. Encode the object using that grid cell and anchor box.

Output Representation

With **2 anchor boxes**, each grid cell predicts two independent output vectors.

The output tensor becomes:

```
3 × 3 × 16
```

since:

```
16 = 2 × 8
```

It can also be viewed as:

```
3 × 3 × 2 × 8
```

where:

- `3 × 3`: grid cells
- `2`: anchor boxes
- `8`: prediction vector for each anchor box

Output Vector

Each anchor box predicts:

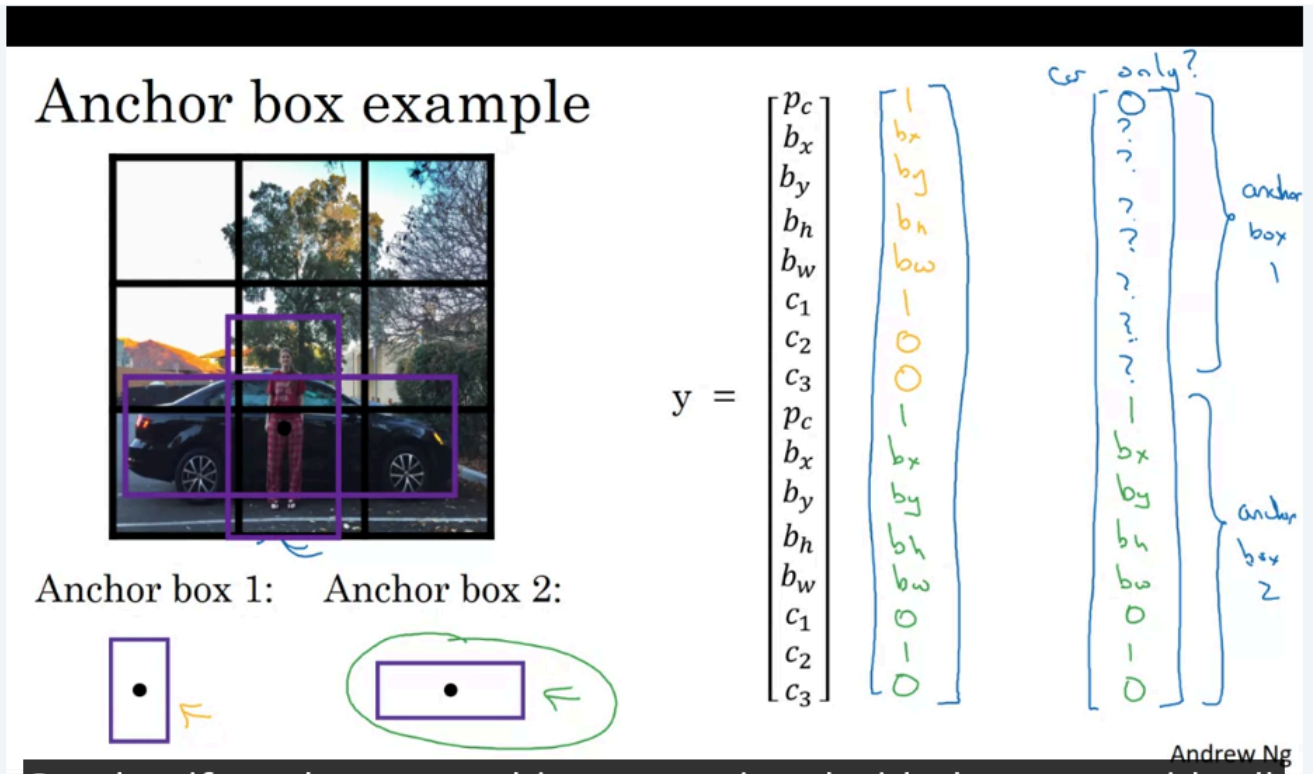
```
[Pc, Bx, By, Bh, Bw, C1, C2, C3]
```

The dimension is **8** because the example contains **3 object classes**.

If the number of object classes increases, the prediction vector becomes larger accordingly.

Key Takeaways

- Without anchor boxes, each object is assigned only to a grid cell.
- With anchor boxes, each object is assigned to a **(grid cell, anchor box)** pair.
- The selected anchor box is the one with the highest IoU with the object's shape.
- Using 2 anchor boxes changes the output from **3 × 3 × 8** to **3 × 3 × 16** (or equivalently **3 × 3 × 2 × 8**).



Anchor Boxes: Complete Example

Encoding Multiple Objects

Suppose a grid cell contains:

- A **pedestrian**
- A **car**

If the pedestrian's shape is closer to **Anchor Box 1**:

```
Anchor Box 1:
[1, Bx, By, Bh, Bw, 1, 0, 0]
```

- `Pc = 1`: object exists
- `Bx, By, Bh, Bw`: pedestrian bounding box
- `C1 = 1`: object class is pedestrian

If the car's shape is closer to **Anchor Box 2**:

```
Anchor Box 2:
[1, Bx, By, Bh, Bw, 0, 1, 0]
```

- `Pc = 1`
- Bounding box describes the car
- `C2 = 1`: object class is car

Thus, one grid cell can encode two different objects using two anchor boxes.

Grid Cell with Only One Object

If the grid cell contains **only a car**:

The Anchor Box 2 prediction remains unchanged:

```
Anchor Box 2:
[1, Bx, By, Bh, Bw, 0, 1, 0]
```

For Anchor Box 1:

```
Anchor Box 1:
[0, ?, ?, ?, ?, ?, ?, ?]
```

where:

- `Pc = 0`: no object assigned
- Remaining values are **don't care** values and are ignored during training.

Limitations

More Objects than Anchor Boxes

If a grid cell contains:

- 3 objects
- but only 2 anchor boxes

the algorithm cannot represent all objects simultaneously.

A predefined tie-breaking rule is typically used.

Multiple Objects Prefer the Same Anchor Box

If two objects:

- belong to the same grid cell
- have similar shapes
- both match the same anchor box

the algorithm again requires a default tie-breaking strategy.

Fortunately, these situations occur infrequently.

Practical Motivation

Originally, anchor boxes were introduced to handle multiple objects in the same grid cell.

However, with a **19 × 19 grid**, two objects rarely share the same cell.

Their greater practical benefit is **model specialization**.

Learning Specialization

Different anchor boxes naturally specialize in different object shapes.

Example:

Anchor Box	Learns to Detect
Tall, narrow	Pedestrians
Wide, short	Cars

This specialization improves detection performance.

Choosing Anchor Boxes

Manual Selection

A simple approach is to manually define several anchor box shapes that cover common object types.

For example:

- Tall and skinny
- Wide and short
- Medium square

Using **5–10 anchor boxes** is common.

Automatic Selection

A more advanced approach is to use **K-means clustering** on the training set bounding boxes.

The algorithm:

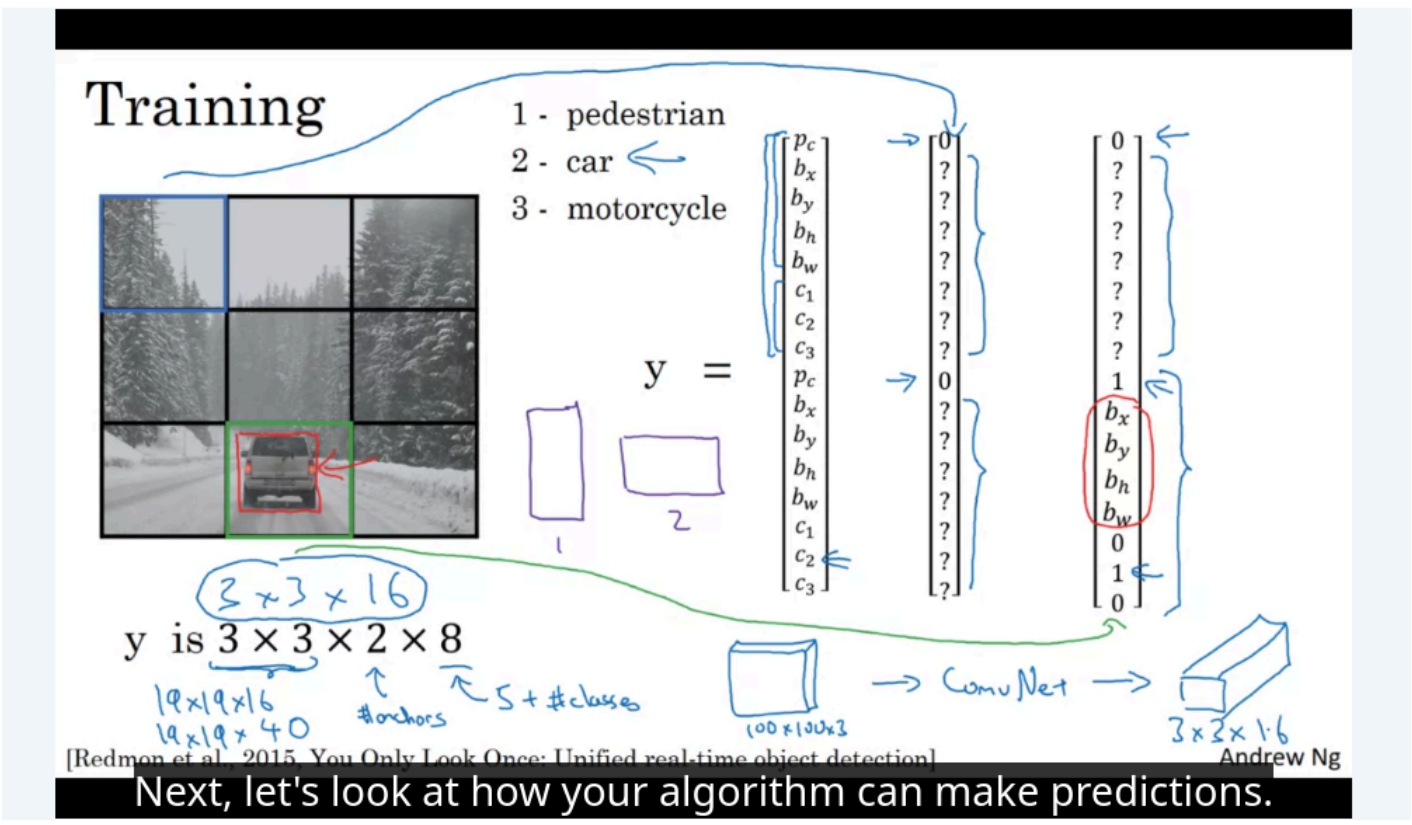
1. Groups objects with similar shapes.
2. Finds representative bounding box sizes.
3. Uses those representatives as anchor boxes.

This produces anchor boxes that better match the dataset distribution.

Key Takeaways

- Each object is assigned to the anchor box with the most similar shape.
- If only one anchor box contains an object, other anchor boxes have `Pc = 0` and don't-care labels.
- Anchor boxes have limitations when too many objects occupy the same grid cell or prefer the same anchor box.
- Their main practical advantage is enabling different prediction heads to specialize in detecting different object shapes.
- Anchor boxes can be chosen manually or automatically using **K-means clustering** on bounding box shapes.

YOLO Algorithm



YOLO: Training Pipeline

Object Classes

Assume the model detects **three object classes**:

- Pedestrians
- Cars
- Motorcycles

The background is represented by:

- `Pc = 0` (no object)

Output Representation

Using:

- **3 × 3 grid**
- **2 anchor boxes**

the output tensor is:

```
3 × 3 × 2 × 8
```

or equivalently:

```
3 × 3 × 16
```

where:

```
8 = 5 + number_of_classes
```

- `5`: `Pc + Bx + By + Bh + Bw`
- `3`: `C1 + C2 + C3`

Constructing the Training Labels

For every grid cell:

1. Determine whether an object exists.
2. Assign the object to the anchor box with the highest IoU.
3. Build the corresponding target vector.

This process is repeated for every grid cell in the image.

Example: Empty Grid Cell

If a grid cell contains no pedestrian, car, or motorcycle:

```
Anchor Box 1:
[0, ?, ?, ?, ?, ?, ?, ?]

Anchor Box 2:
[0, ?, ?, ?, ?, ?, ?, ?]
```

where:

- `Pc = 0`
- Remaining values are **don't cares**.

Most grid cells in an image are typically empty.

Example: Grid Cell Containing a Car

Suppose:

- The ground-truth car bounding box is slightly wider than it is tall.
- It has a higher IoU with **Anchor Box 2**.

Then:

```
Anchor Box 1:
[0, ?, ?, ?, ?, ?, ?, ?]

Anchor Box 2:
[1, Bx, By, Bh, Bw, 0, 1, 0]
```

where:

- `Pc = 1`
- `Bx, By, Bh, Bw` encode the car bounding box.

- `C2 = 1` indicates the object is a car.

Training Procedure

For every grid position:

- Generate one **16-dimensional target vector**.
- Repeat this for all **3 × 3 = 9 grid cells**.

The collection of these vectors forms the complete training label:

```
3 × 3 × 16
```

Practical YOLO Output

The 3 × 3 example is used only for illustration.

Real implementations commonly use:

```
19 × 19 × 16
```

with **2 anchor boxes**.

If **5 anchor boxes** are used:

```
19 × 19 × (5 × 8)
= 19 × 19 × 40
```

ConvNet Training

The ConvNet takes an input image, for example:

```
100 × 100 × 3
```

and directly predicts the output tensor:

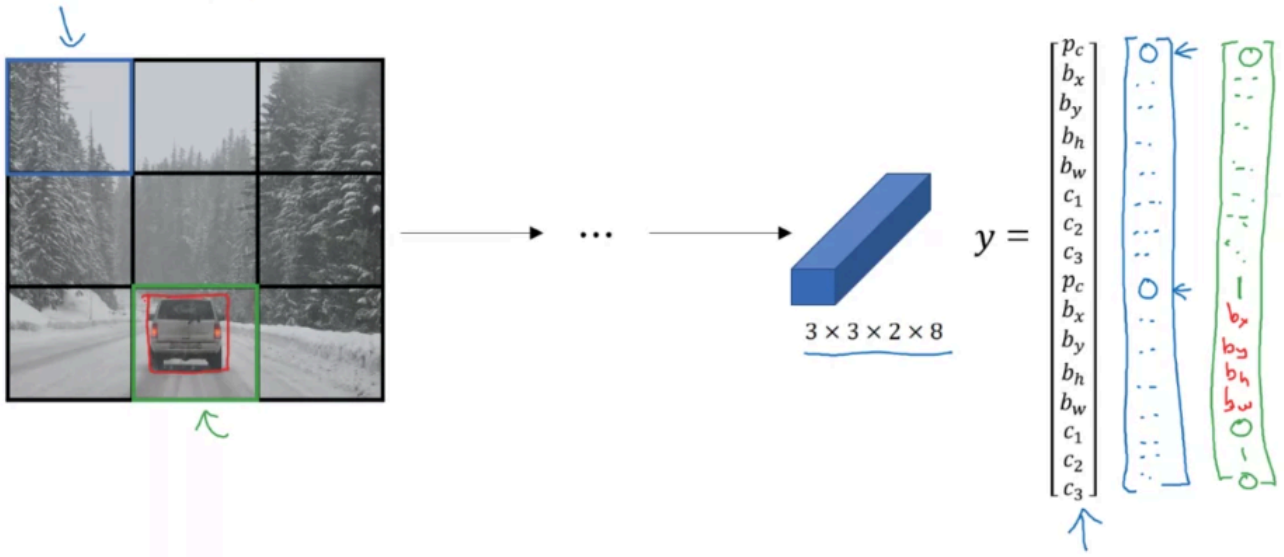
```
3 × 3 × 16
```

or equivalently:

```
3 × 3 × 2 × 8
```

In practice, the output is typically larger (e.g., **19 × 19 × 40**) depending on the grid size and number of anchor boxes.

Making predictions



Andrew Ng

So that's how the neural network will make predictions.

Outputting the non-max supressed outputs

- For each grid call, get 2 predicted bounding boxes.

Andrew Ng

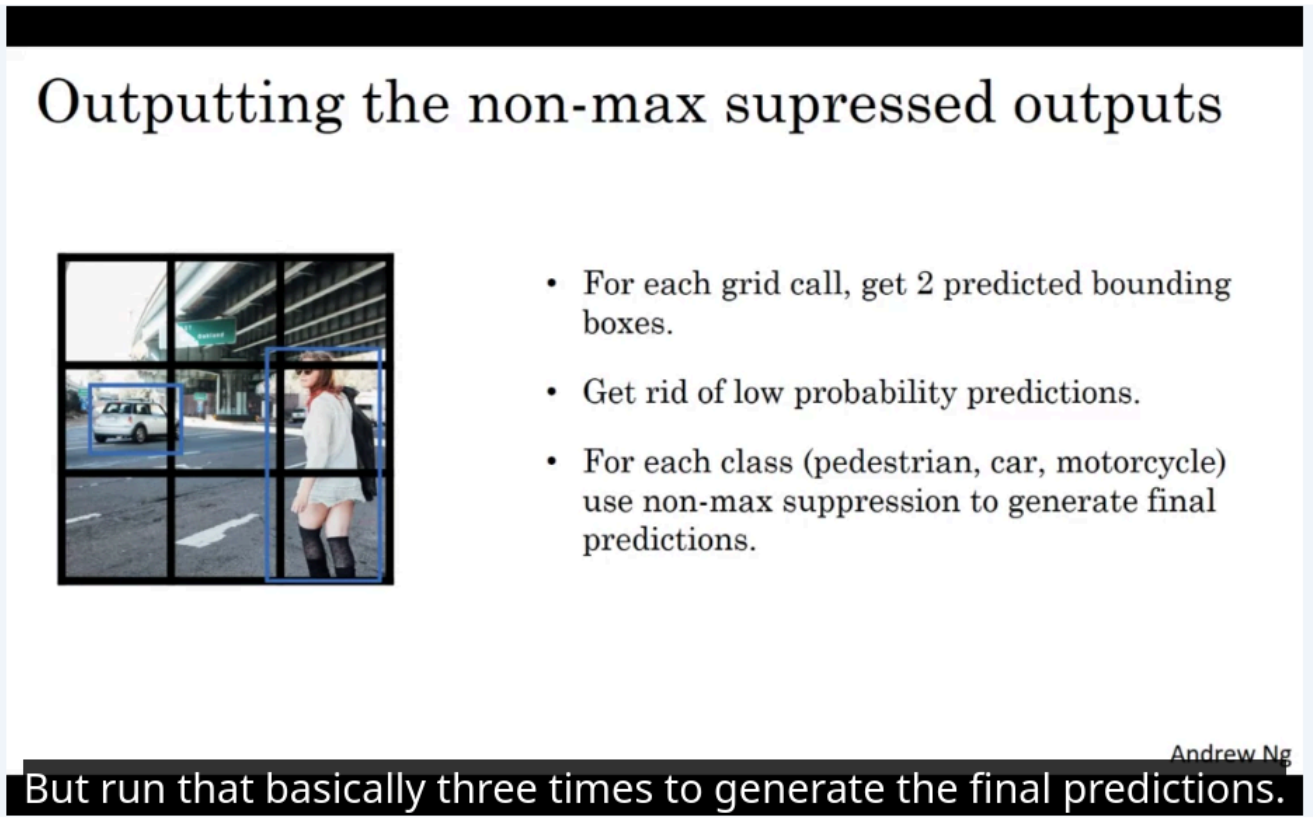
So let's say, those are the bounding boxes you get.

Outputting the non-max supressed outputs

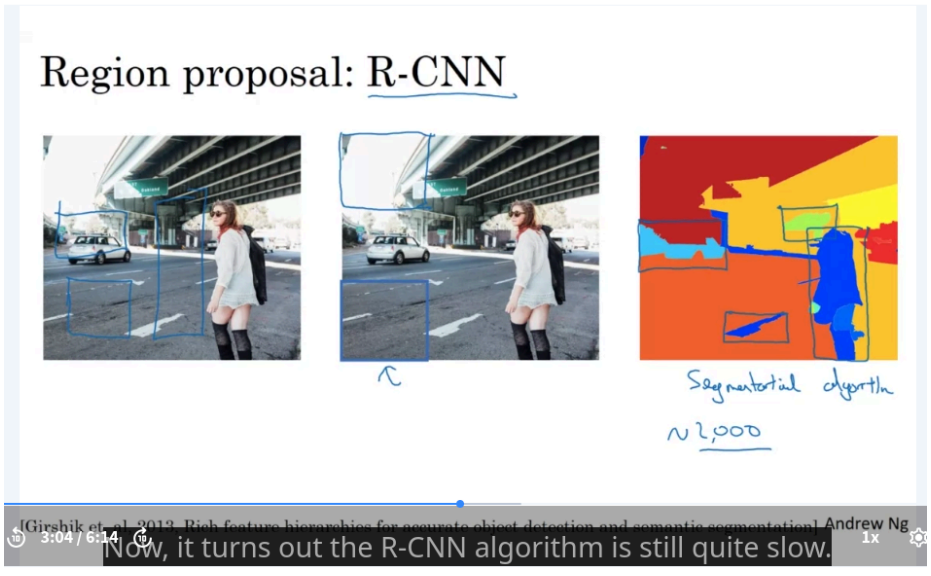
- For each grid call, get 2 predicted bounding boxes.
- Get rid of low probability predictions.

Andrew Ng

So get rid of those.



Region Proposals (Optional)



Region Proposals and R-CNN

Motivation

The sliding window approach evaluates **every possible region** in an image.

Although it can be implemented convolutionally, it still processes many regions that clearly contain no objects.

Examples:

- Empty background regions
- Large blank areas
- Uninformative image patches

This results in unnecessary computation.

Region Proposal Idea

Instead of scanning every possible window:

1. Identify regions that are likely to contain objects.
2. Run the CNN classifier only on those selected regions.

This significantly reduces the number of candidate windows.

R-CNN (Regions with CNN Features)

R-CNN was proposed by:

- Ross Girshick
- Jeff Donahue
- Trevor Darrell
- Jitendra Malik

The key idea is:

> Generate object proposals first, then classify only those proposals using a CNN.

Region Proposal Generation

R-CNN uses a **segmentation algorithm** to identify meaningful image regions (blobs).

The segmentation algorithm groups pixels into candidate object regions.

Example:

- A blob covering a pedestrian
- A blob covering a car
- Other blobs covering potential objects

Each blob is converted into a bounding box for classification.

R-CNN Pipeline

```
Input Image
      ↓
Segmentation Algorithm
      ↓
Generate Candidate Blobs
      ↓
Create Bounding Boxes
      ↓
Run CNN Classifier on Each Proposal
      ↓
Object Predictions
```

Number of Proposals

Instead of evaluating every sliding window, R-CNN typically:

- Generates approximately **2000 region proposals**.
- Runs the CNN only on these 2000 candidate regions.

This is much more efficient than exhaustively scanning the entire image.

Advantages

- Avoids evaluating obvious background regions.
- Focuses computation on likely object locations.
- Reduces the number of candidate windows compared to sliding windows.

Flexibility

Region proposals can have different shapes and sizes:

- Square regions
- Tall and narrow regions (e.g., pedestrians)

- Wide regions (e.g., cars)
- Multiple scales

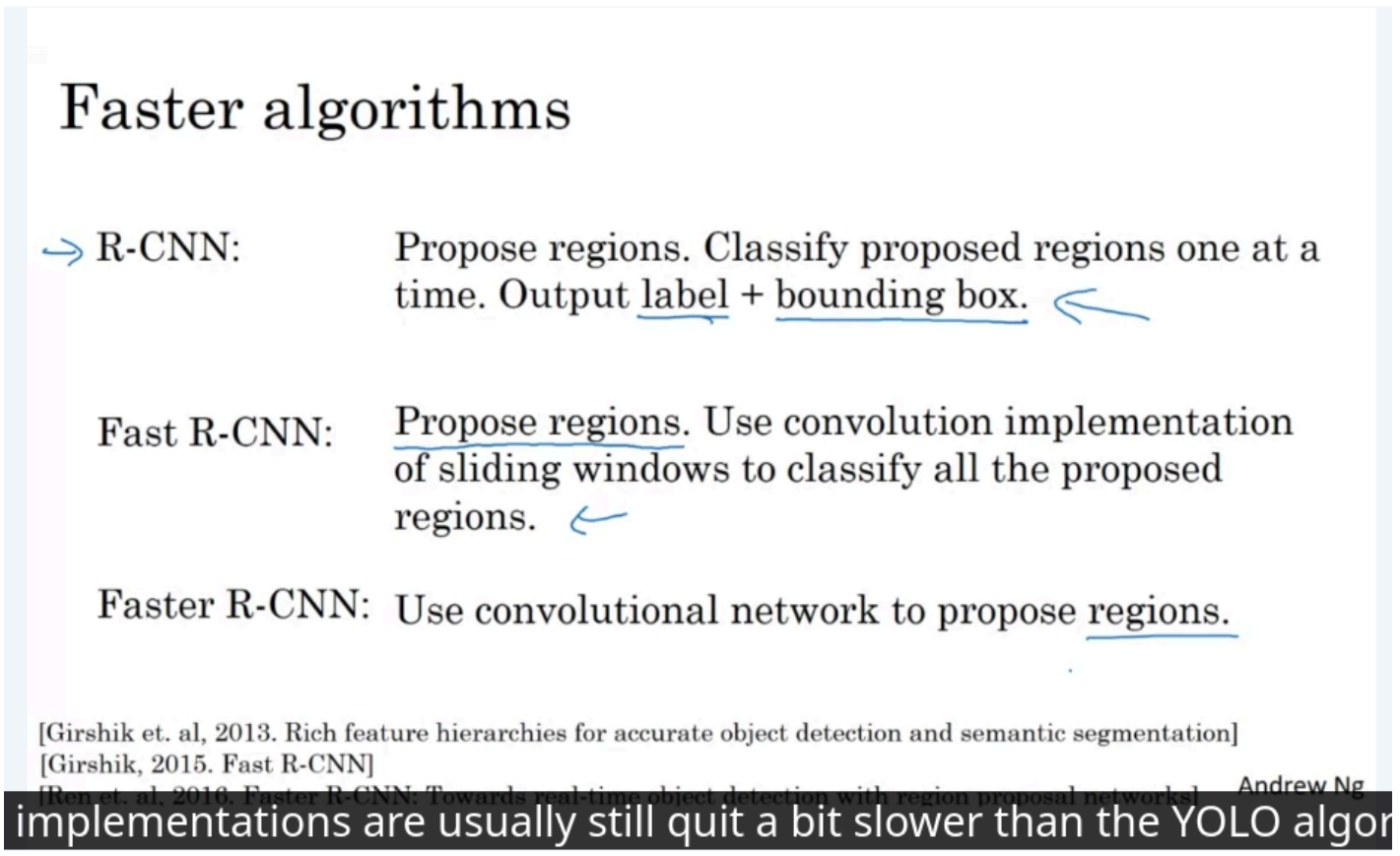
This makes R-CNN more flexible than using only fixed-size sliding windows.

Limitation

Although R-CNN greatly reduces the number of evaluated regions, it is **still relatively slow** because:

- It generates thousands of region proposals.
- The CNN must be executed separately on each proposed region.

This limitation motivated later improvements such as **Fast R-CNN** and **Faster R-CNN**.



R-CNN Improvements

Basic R-CNN Pipeline

R-CNN performs object detection in two stages:

```
Input Image
      ↓
Region Proposal Algorithm
      ↓
Candidate Regions
      ↓
CNN Classifier (one region at a time)
      ↓
Class Label + Refined Bounding Box
```

For each proposed region, the model predicts:

- Object class (car, pedestrian, motorcycle, etc.)
- A refined bounding box (`Bx, By, Bh, Bw`)

R-CNN does **not simply use the proposed bounding box**. It learns to adjust it for more accurate localization.

Limitation of R-CNN

The main drawback is **speed**.

Although region proposals reduce the number of candidate windows, the CNN still processes each proposal independently, making inference computationally expensive.

Fast R-CNN

Main Improvement

Fast R-CNN replaces repeated per-region computation with a **convolutional implementation**.

Instead of:

```
Region 1 → CNN
Region 2 → CNN
Region 3 → CNN
...
```

the image is processed through the convolutional network once, and region features are extracted from the shared feature map.

Benefits

- Eliminates redundant CNN computations.
- Significantly speeds up R-CNN.
- Produces the same type of outputs:
  - Object class
  - Refined bounding box.

Faster R-CNN

Remaining Bottleneck

Fast R-CNN is still limited by the **region proposal (segmentation/clustering) step**, which remains slow.

Main Improvement

Faster R-CNN replaces the traditional region proposal algorithm with a **Convolutional Neural Network**.

```
Input Image
      ↓
CNN
      ├── Region Proposal Network (RPN)
      └── Object Detection
```

The CNN automatically proposes candidate regions, making the entire pipeline much faster than Fast R-CNN.

Speed Comparison

Algorithm	Main Idea	Speed
R-CNN	Region proposals + CNN on each proposal	Slow
Fast R-CNN	Shared convolutional computation	Faster
Faster R-CNN	CNN-based region proposals (RPN)	Faster than Fast R-CNN
YOLO	Single-stage detection (predict everything at once)	Typically the fastest

According to the lecture, most Faster R-CNN implementations are still slower than YOLO.

Region Proposal vs. YOLO

Region Proposal Methods

- First generate candidate regions.
- Then classify each region.
- Two-stage detection pipeline.

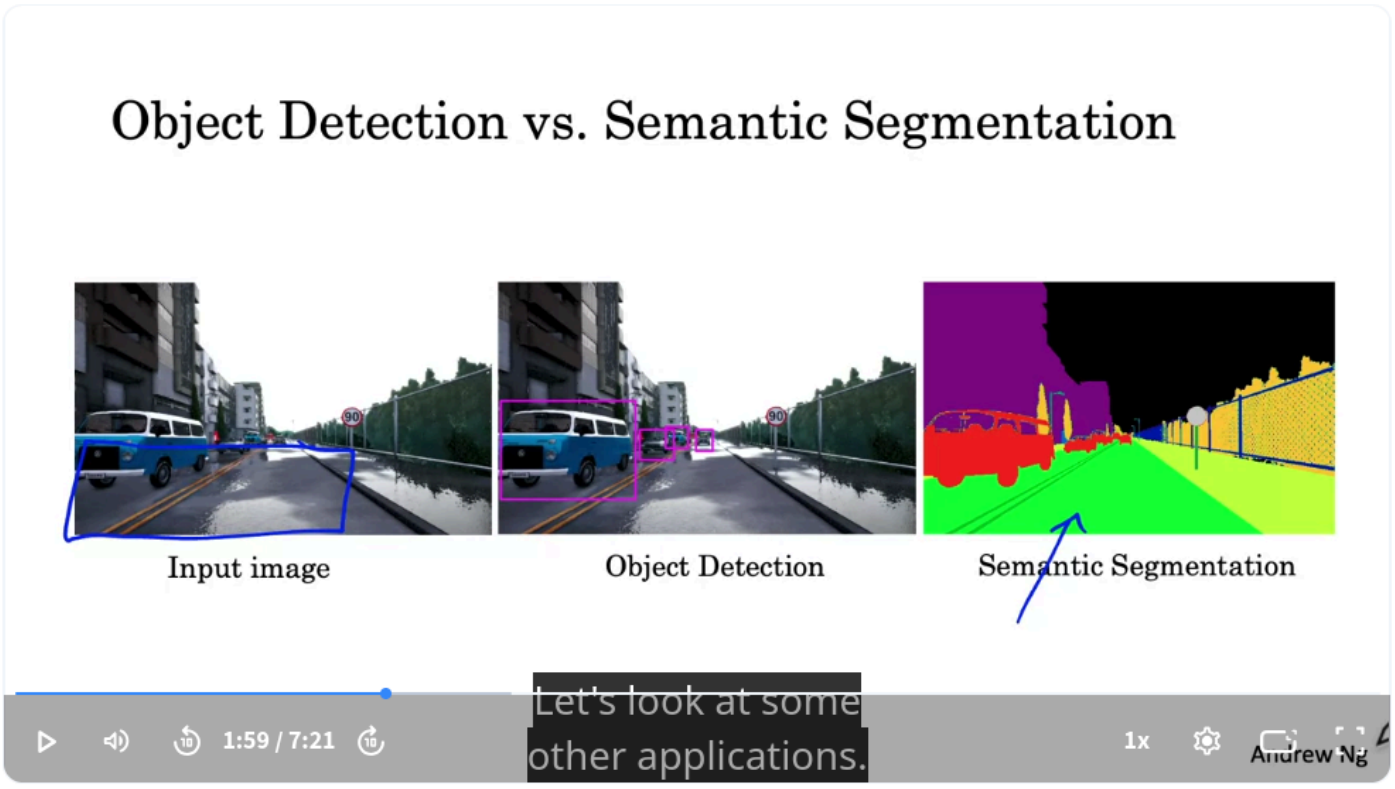
YOLO

- Predicts object locations and classes in a single forward pass.
- No separate region proposal stage.
- End-to-end object detection.

Key Takeaways

- **R-CNN:** uses region proposals and classifies each region individually.
- **Fast R-CNN:** shares convolutional computation to improve efficiency.
- **Faster R-CNN:** introduces a CNN-based Region Proposal Network (RPN) for faster proposal generation.
- **YOLO:** performs object detection in a single stage and is generally faster than the R-CNN family.
- Region proposal methods remain influential, but single-stage detectors like YOLO offer a simpler and more efficient detection pipeline.

Semantic Segmentation with U-Net



Semantic Segmentation

From Recognition to Segmentation

Computer vision tasks can be viewed as increasing levels of detail:

Task	Output
Object Recognition	Identify what object is in the image (e.g., cat or not).
Object Detection	Identify the object and predict a bounding box around it.
Semantic Segmentation	Classify every pixel in the image and produce a precise object outline.

What is Semantic Segmentation?

Semantic segmentation assigns a **class label to every pixel** in an image.

Instead of predicting a coarse bounding box, the model determines exactly which pixels belong to each object or region.

```
Input Image
        ↓
Semantic Segmentation Model
        ↓
Pixel-wise Class Labels
```

Example: Self-Driving Cars

Object Detection

Object detection identifies surrounding vehicles by predicting bounding boxes.

```
Car
 ┌────────────┐
 │            │
 │   Vehicle  │
 │            │
 └────────────┘
```

This provides the object's approximate location.

Semantic Segmentation

Semantic segmentation labels every pixel individually.

Example classes:

- Road
- Car
- Sidewalk
- Building
- Sky

Instead of drawing a box around the road, the algorithm identifies **every pixel that belongs to the drivable surface**.

Pixel-Level Prediction

For each pixel, the model predicts:

```
Pixel → Class
```

Example:

Pixel Region	Predicted Class
Road surface	Drivable road
Vehicle	Car
Sky	Sky
Building	Building

The final output is a dense map where every pixel has a semantic label.

Advantages

Compared with object detection:

- Produces precise object boundaries.
- Provides complete scene understanding.
- Can identify irregularly shaped regions (e.g., roads, sidewalks).

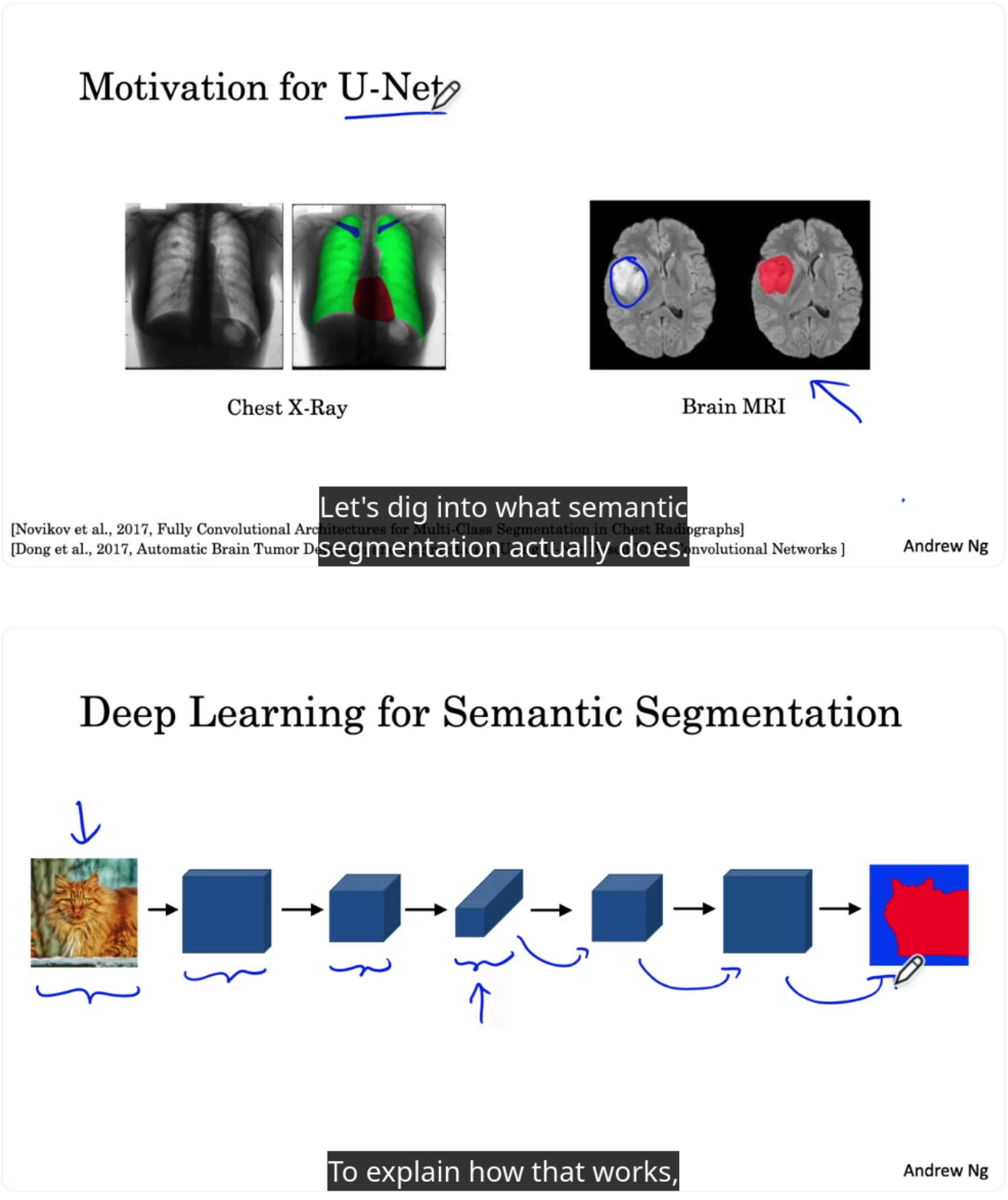
Application: Autonomous Driving

One important application is **drivable area estimation**.

Instead of detecting the road with a bounding box, semantic segmentation determines:

- Which pixels are safe to drive on.
- Which pixels correspond to obstacles.

This enables a self-driving car to make more accurate navigation decisions.



Semantic Segmentation Applications

Medical Imaging

Semantic segmentation is widely used in medical diagnosis and surgical planning.

Chest X-ray Segmentation

The model segments different anatomical structures, such as:

- Lungs
- Heart
- Clavicles (collarbones)

Benefits:

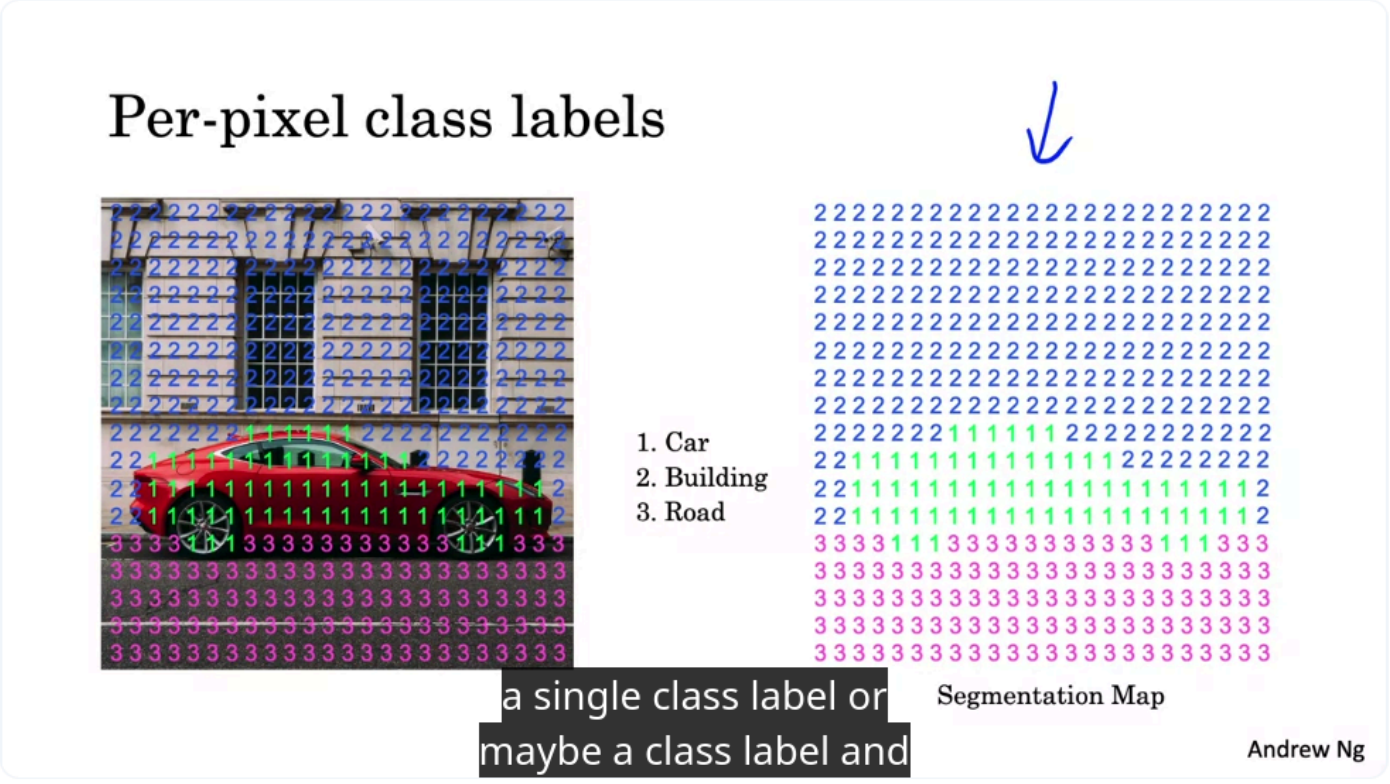
- Easier detection of abnormalities
- Improved disease diagnosis
- Better surgical planning

Brain Tumor Segmentation

Given a brain MRI:

- The model automatically identifies tumor pixels.
- Manual segmentation by radiologists is time-consuming.
- Automatic segmentation saves time and provides valuable information for surgery planning.

Pixel-wise Classification



Binary Segmentation

If the goal is only to detect cars:

Assign two labels:

```
1 → Car
0 → Background (Not Car)
```

The model predicts one label for **every pixel**.

Multi-class Segmentation

More detailed segmentation can assign multiple classes:

Class	Label
Car	1
Building	2
Road/Ground	3

Each pixel is assigned exactly one semantic class.

Output of Semantic Segmentation

Unlike classification or object detection:

- **Classification:** outputs one class label.
- **Object Detection:** outputs class labels + bounding boxes.
- **Semantic Segmentation:** outputs an entire matrix of pixel labels.

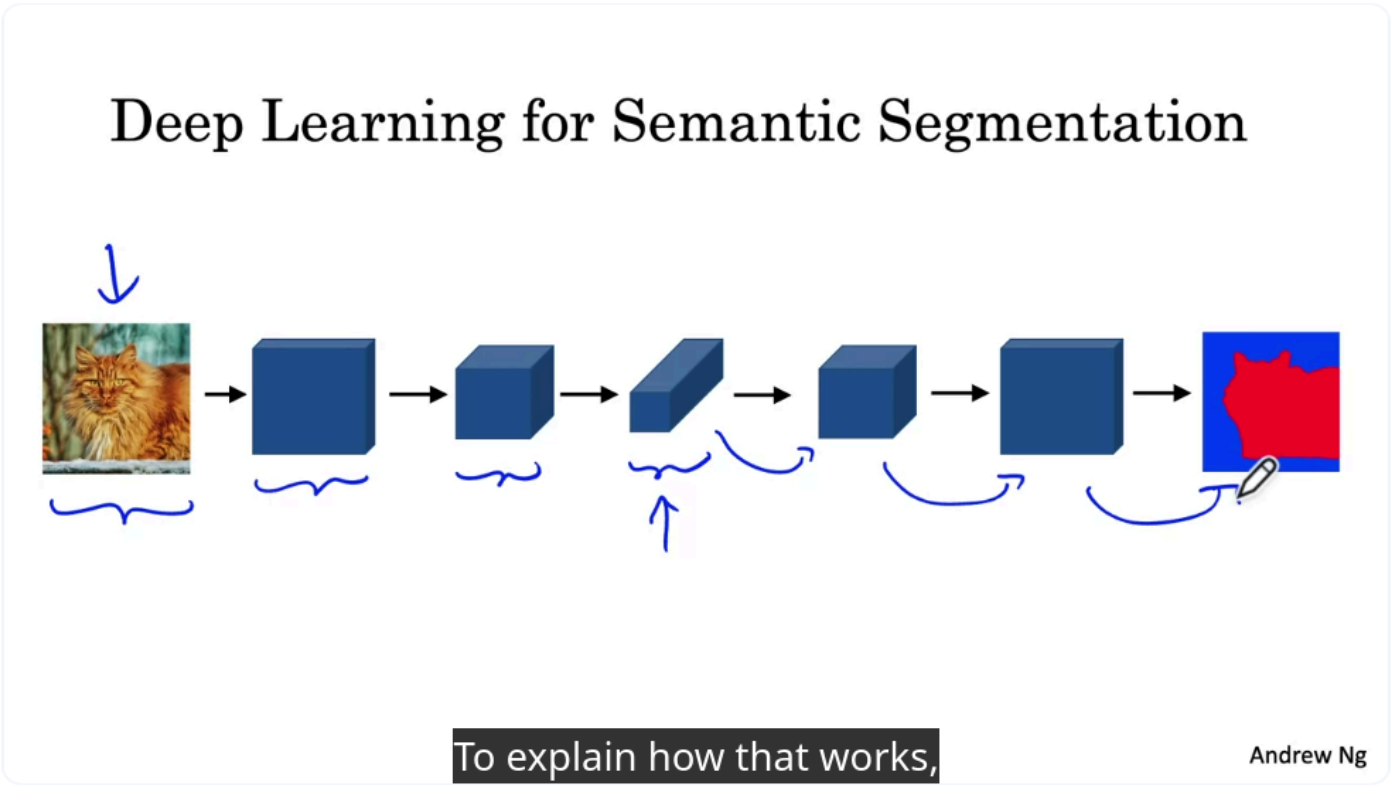
```
Input Image
      ↓
Semantic Segmentation Network
```

```
        ↓
Pixel-wise Label Matrix
```

The output has the same spatial resolution as the desired segmentation map.

U-Net Architecture

A common architecture for semantic segmentation is **U-Net**.



Encoder

The first half behaves like a standard CNN:

- Extract features
- Reduce image height and width
- Increase feature abstraction

```
Input
  ↓
Conv
  ↓
Pooling
  ↓
Smaller Feature Maps
```

Decoder

Instead of continuing to shrink the feature maps, U-Net expands them back to the original image size.

During decoding:

- Height and width increase.
- Number of channels gradually decreases.

```
Small Feature Maps
        ↓
Upsampling
        ↓
Larger Feature Maps
        ↓
Segmentation Map
```

The final output is a full-resolution pixel-wise prediction.

Key Operation: Transpose Convolution

A crucial component of U-Net is **transpose convolution**.

Purpose:

- Increase the spatial dimensions of feature maps.
- Convert compact feature representations into full-size segmentation outputs.

Instead of:

```
64 × 64
    ↓
32 × 32
```

transpose convolution performs:

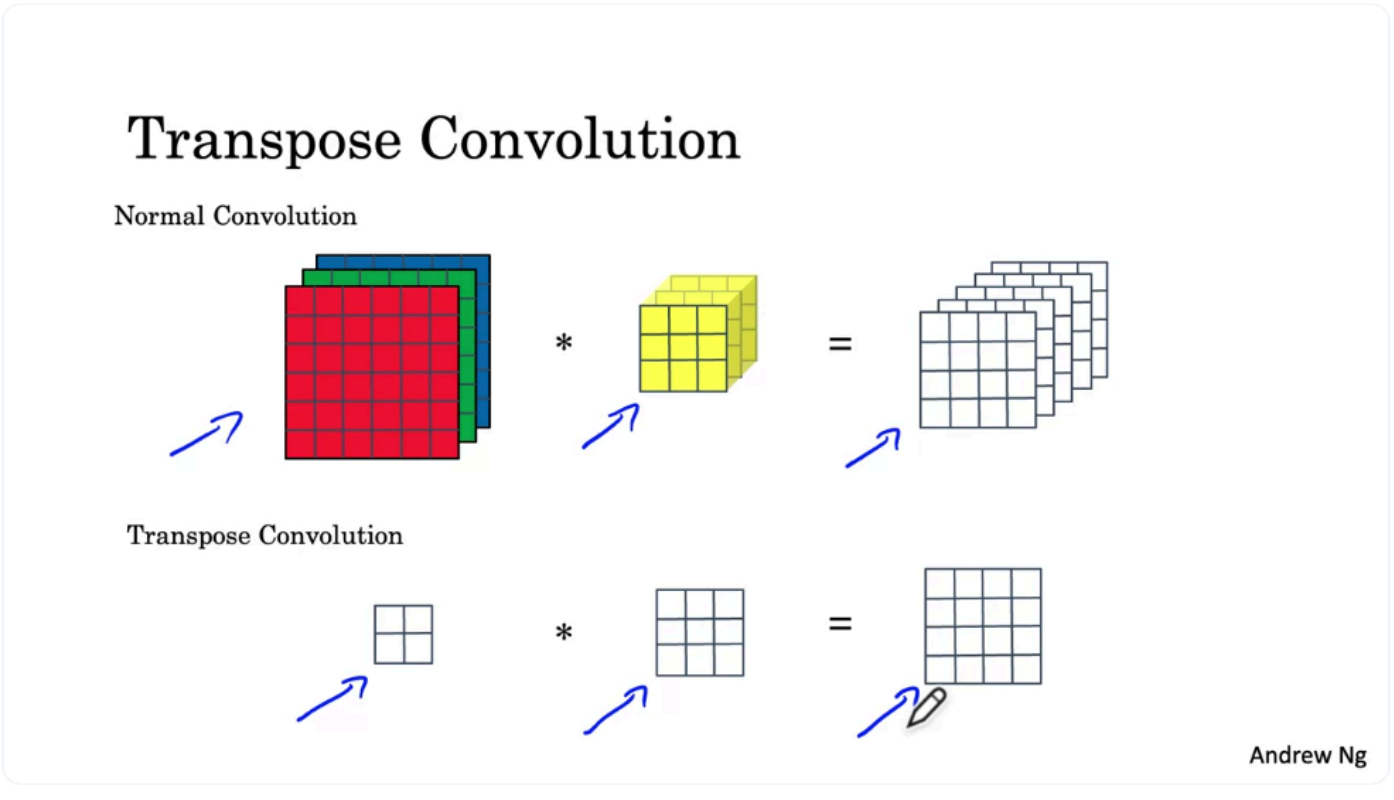
```
32 × 32
    ↓
64 × 64
```

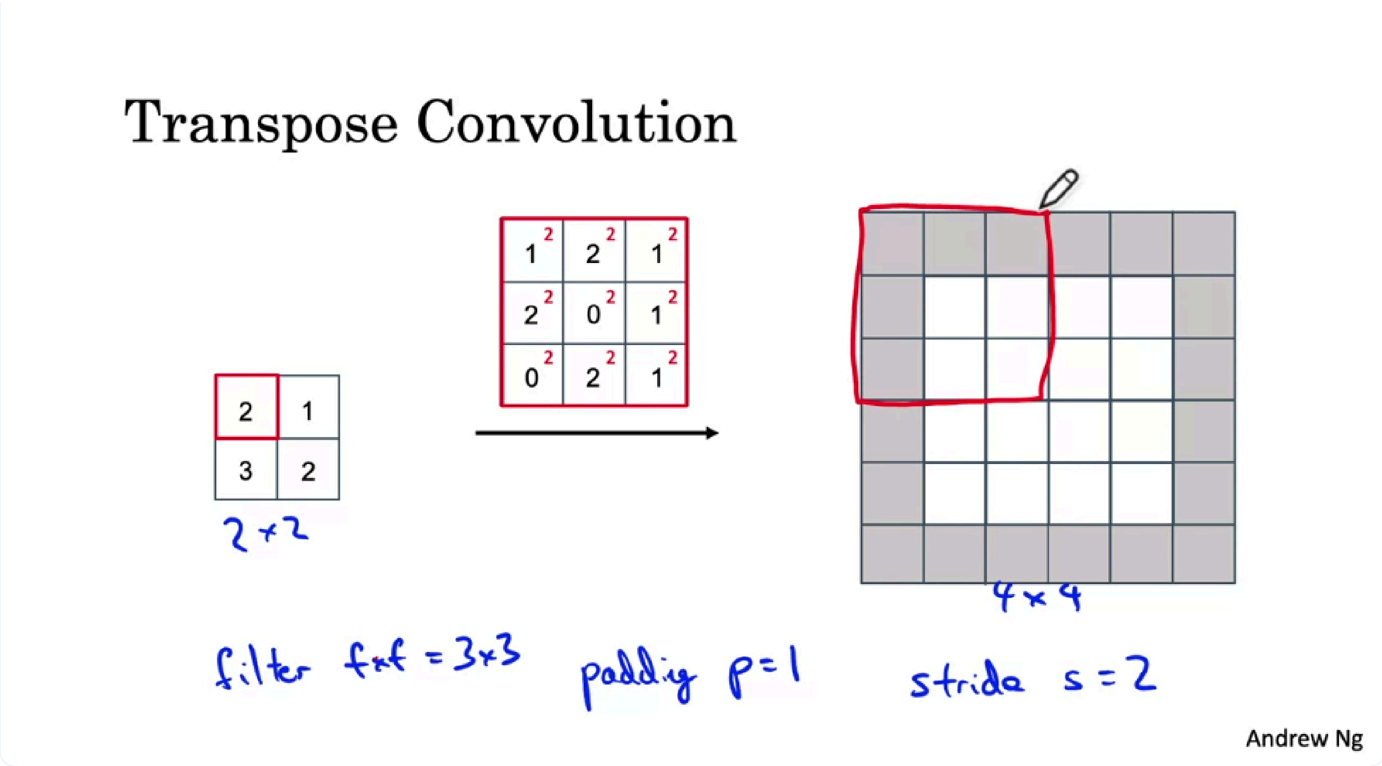
This operation is applied multiple times in the decoder to progressively recover image resolution.

Key Takeaways

- Semantic segmentation predicts a class label for **every pixel**.
- Medical applications include:
  - Chest X-ray anatomy segmentation
  - Brain tumor segmentation
- Segmentation can be:
  - **Binary:** object vs. background
  - **Multi-class:** car, building, road, etc.
- The output is a **matrix of pixel labels**, not a single prediction or bounding box.
- **U-Net** is a popular architecture that:
  - Encodes images into compact feature maps.
  - Decodes them back into full-resolution segmentation maps.
- **Transpose convolution** is the key operation that enlarges feature maps during the decoding process.

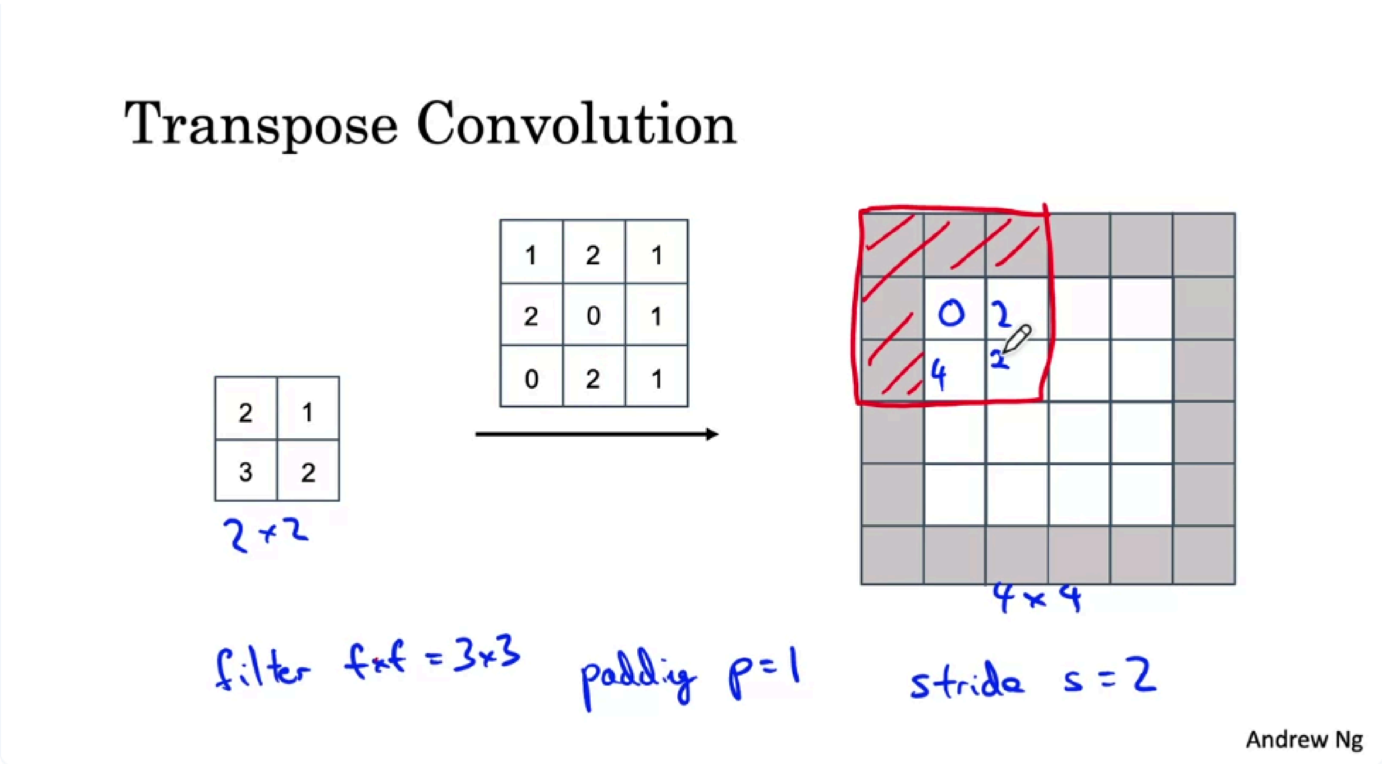
Transpose Convolutions





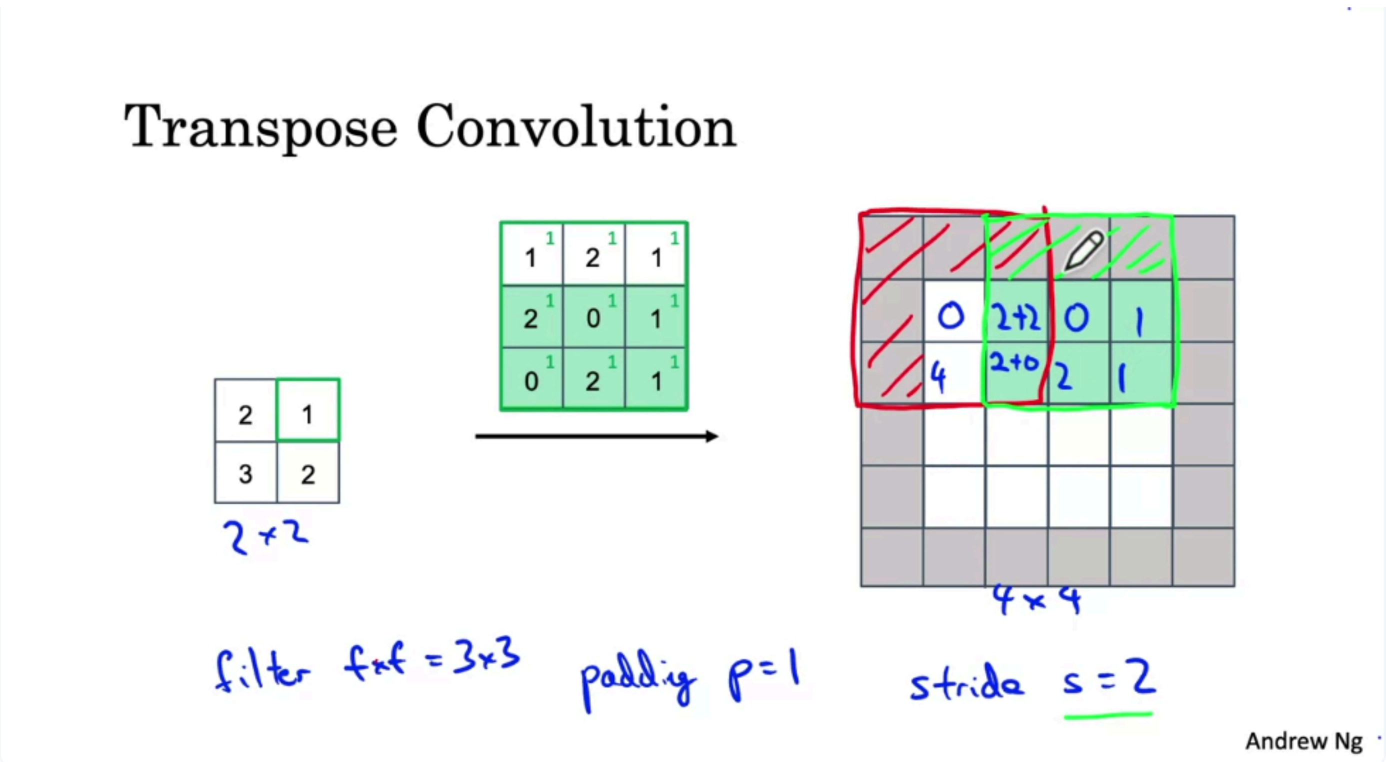
Transpose Convolutions

Save note



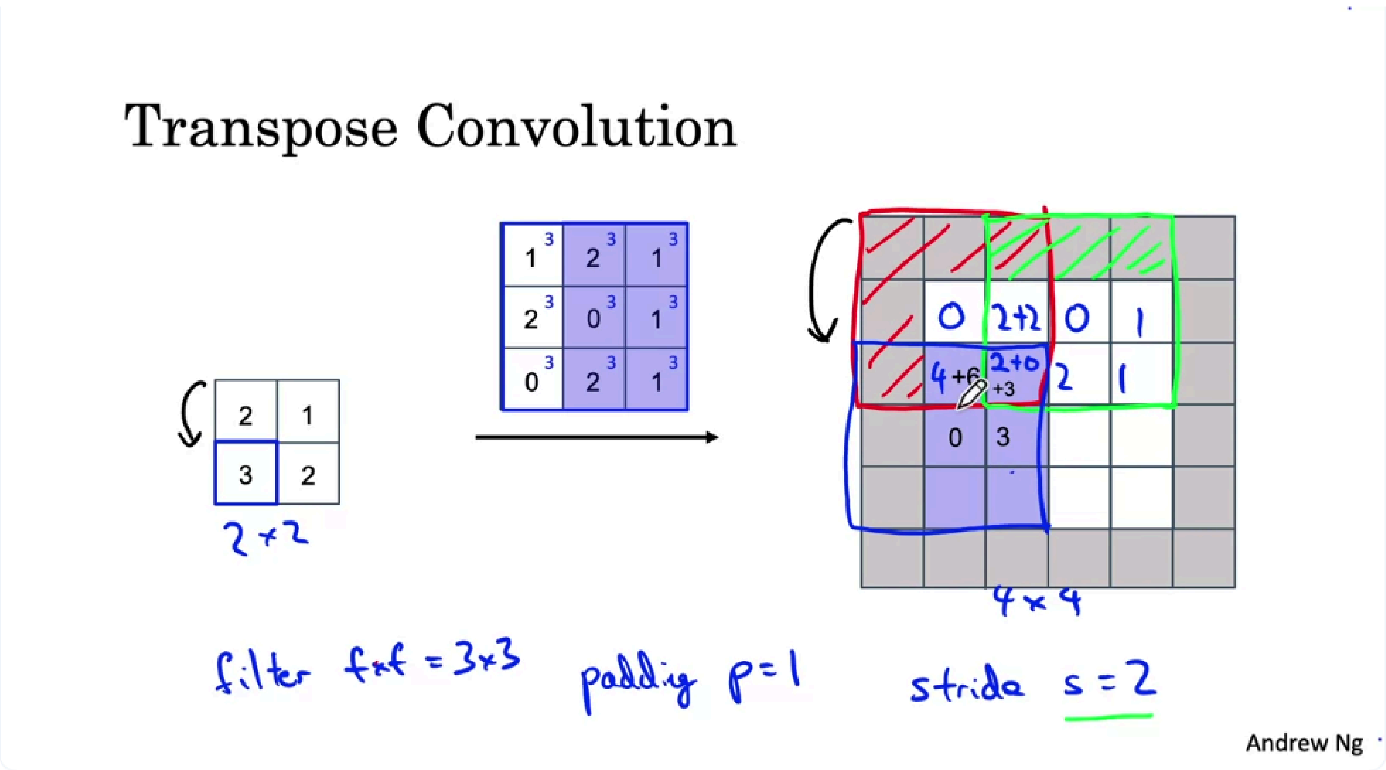
Transpose Convolutions

Save note



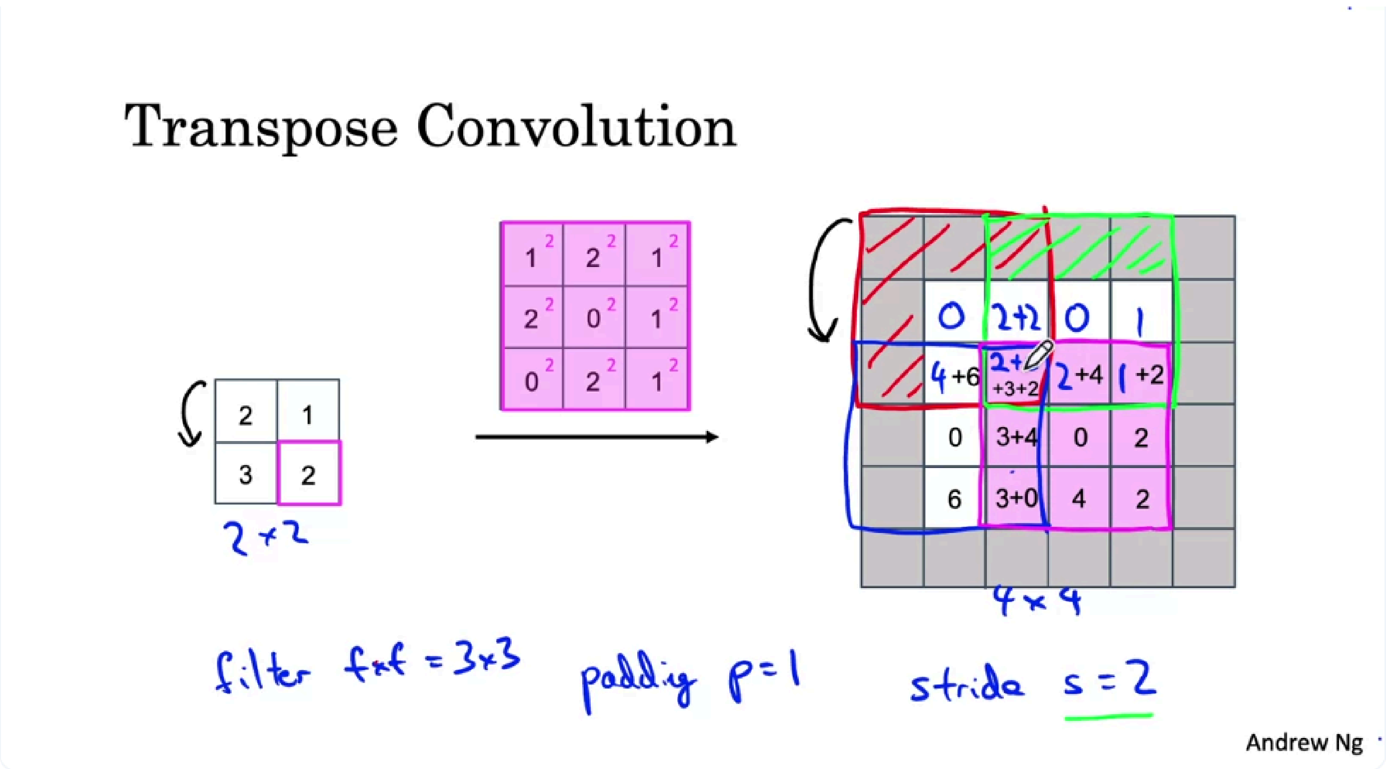
Transpose Convolutions

Save note



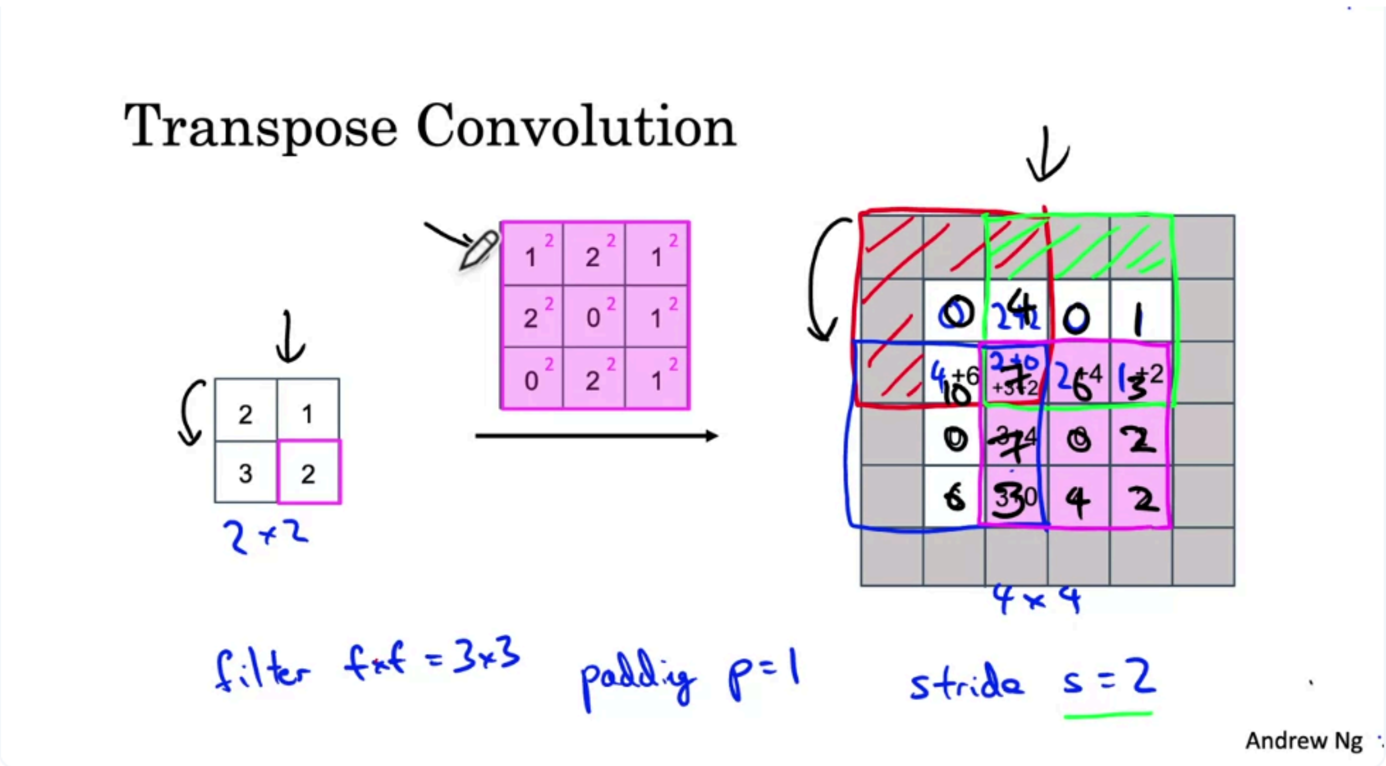
Transpose Convolutions

Save note



Transpose Convolutions

Save note



Transpose Convolutions

Save note

Transpose Convolution

Purpose

Transpose convolution is used to **increase the spatial dimensions** of feature maps.

Example:

```
Input:  2 × 2
        ↓
Transpose Convolution
        ↓
Output: 4 × 4
```

It is a key building block of the **U-Net decoder**, allowing the network to reconstruct full-resolution segmentation maps.

Standard Convolution vs. Transpose Convolution

Standard Convolution	Transpose Convolution
Reduces spatial dimensions	Increases spatial dimensions
Example: 6 × 6 → 4 × 4	Example: 2 × 2 → 4 × 4
Filter slides over the input	Filter is projected onto the output

Example Configuration

Input:

```
2 × 2
```

Filter:

```
3 × 3
```

Hyperparameters:

```
Filter size (f) = 3
Padding (p) = 1
Stride (s) = 2
```

Output:

```
4 × 4
```

Core Idea

Instead of sliding the filter over the input as in standard convolution:

- Take one input value at a time.
- Multiply the entire filter by that value.
- Place the resulting weighted filter onto the output feature map.

Overlapping regions are **summed together**.

Step 1: Process First Input Element

Suppose the upper-left input value is:

```
2
```

Operations:

1. Multiply every filter element by `2`.

2. Place the resulting `3 × 3` block onto the output.
3. Ignore values that fall into the padding area.

Result:

```
Input value
      ↓
2 × Filter
      ↓
Place weighted filter onto output
```

Step 2: Process Next Input Element

Suppose the next input value is:

```
1
```

Operations:

1. Multiply the filter by `1`.
2. Because `stride = 2`, shift the placement by **2 positions**.
3. If the new block overlaps an existing block, **add the values instead of replacing them**.

```
Existing value + New value
```

This accumulation is the key operation of transpose convolution.

Step 3: Repeat for All Input Elements

For every input value:

- Multiply the filter by that value.
- Shift according to the stride.
- Add overlapping values.

Eventually, every input element contributes to the final output feature map.

Final Output

After all weighted filters have been placed:

- Sum all overlapping contributions.
- The resulting matrix becomes the final enlarged feature map.

```
2 × 2 Input
      ↓
Multiply filter by each input value
      ↓
Place weighted filters on output
      ↓
Add overlapping values
      ↓
4 × 4 Output
```

Why Use Transpose Convolution?

There are many ways to enlarge a feature map.

Transpose convolution is popular because:

- The filter weights are **learnable**.
- The network learns how to perform upsampling automatically.

- It produces good results in semantic segmentation architectures such as U-Net.

Role in U-Net

U-Net consists of:

Encoder

```
Input Image
      ↓
Convolution
      ↓
Pooling
      ↓
Smaller Feature Maps
```

Decoder

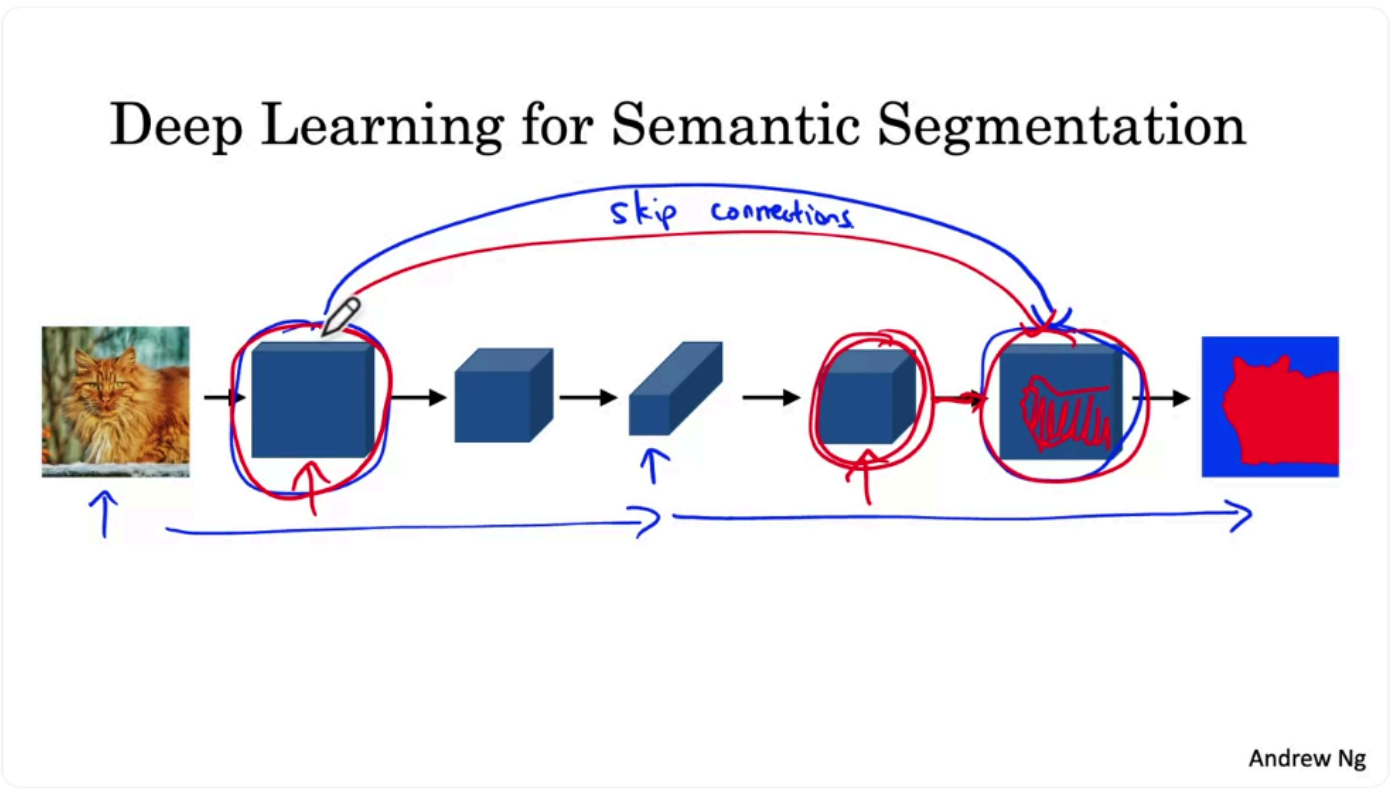
```
Small Feature Maps
      ↓
Transpose Convolution
      ↓
Larger Feature Maps
      ↓
Segmentation Map
```

Multiple transpose convolution layers are applied to progressively recover the original image resolution.

Key Takeaways

- Transpose convolution **upsamples** feature maps by increasing their spatial dimensions.
- Each input value multiplies the entire filter and projects it onto the output.
- **Overlapping regions are summed**, not overwritten.
- Typical hyperparameters include filter size, padding, and stride, just like standard convolution.
- It is the fundamental operation used in the **decoder of U-Net** to generate pixel-wise semantic segmentation outputs.

U-Net Architecture Intuition



U-Net Architecture: High-Level Intuition

Overall Structure

U-Net consists of two main parts:

```
Input Image
      ↓
Encoder (Downsampling)
      ↓
Compressed Feature Representation
      ↓
Decoder (Upsampling)
      ↓
Segmentation Map
```

The goal is to produce a **pixel-wise segmentation map** with the same resolution as the input image.

Encoder (Contracting Path)

The first half of U-Net uses **standard convolutions**.

Its purpose is to:

- Extract high-level features.
- Gradually reduce the spatial dimensions (height and width).
- Increase feature abstraction.

```
Input Image
      ↓
Conv + Pooling
      ↓
Smaller Feature Maps
      ↓
High-Level Representation
```

Characteristics

- **Height and width decrease.**
- **Number of channels increases.**
- Spatial details become less precise.
- Semantic understanding becomes stronger.

For example, the encoder may recognize:

> "There is probably a cat in the lower-right region."

but it no longer knows the exact pixel boundaries.

Decoder (Expanding Path)

The second half of U-Net uses **transpose convolutions**.

Its purpose is to:

- Increase the spatial resolution.
- Gradually reconstruct a full-size image.
- Produce a segmentation map.

```
Compressed Features
        ↓
Transpose Convolution
        ↓
Larger Feature Maps
        ↓
Pixel-wise Prediction
```

Characteristics

- **Height and width increase.**
- **Number of channels decreases.**
- Feature maps are expanded back to the original image size.

Problem Without Skip Connections

After multiple downsampling operations:

- The network retains high-level semantic information.
- Much of the fine spatial detail is lost.

For example, it may know:

> "A cat exists in the lower-right area."

but not:

> "Which exact pixels belong to the cat?"

Skip Connections

U-Net solves this problem by introducing **skip connections**.

```
Encoder                     Decoder

High-resolution features ──────────► Upsampling layer
```

Instead of relying only on compressed features, the decoder also receives feature maps directly from earlier encoder layers.

Why Skip Connections Help

The decoder combines **two complementary types of information**:

1. High-Level Context

From deeper layers:

- Object identity
- Global semantic understanding
- Approximate object location

Example:

> "There is a cat in the right side of the image."

2. Fine-Grained Spatial Details

From early layers via skip connections:

- Texture
- Edges
- Pixel-level details
- High spatial resolution

Example:

> "These pixels contain fur-like patterns."

Information Fusion

The decoder integrates both sources:

```
High-Level Context
        +
Fine Spatial Details
        ↓
Accurate Pixel-wise Segmentation
```

This enables precise object boundaries while preserving semantic understanding.

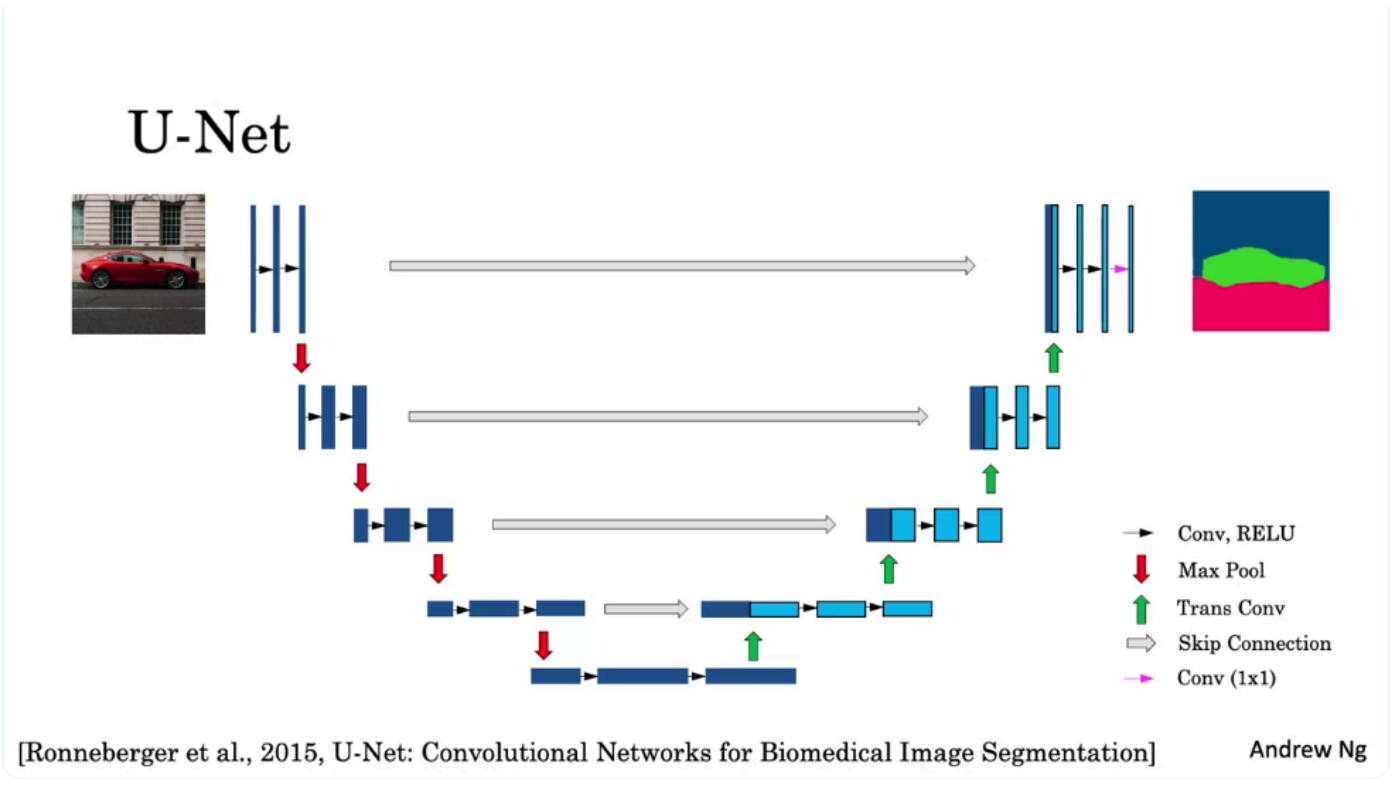
U-Net Workflow

```
Input Image
      ↓
Encoder
(Extract semantic features)
      ↓
Compressed Representation
      ↓
Transpose Convolution
      ↓
Combine with Skip Connection
      ↓
Upsampled Feature Map
      ↓
Repeat
      ↓
Segmentation Map
```

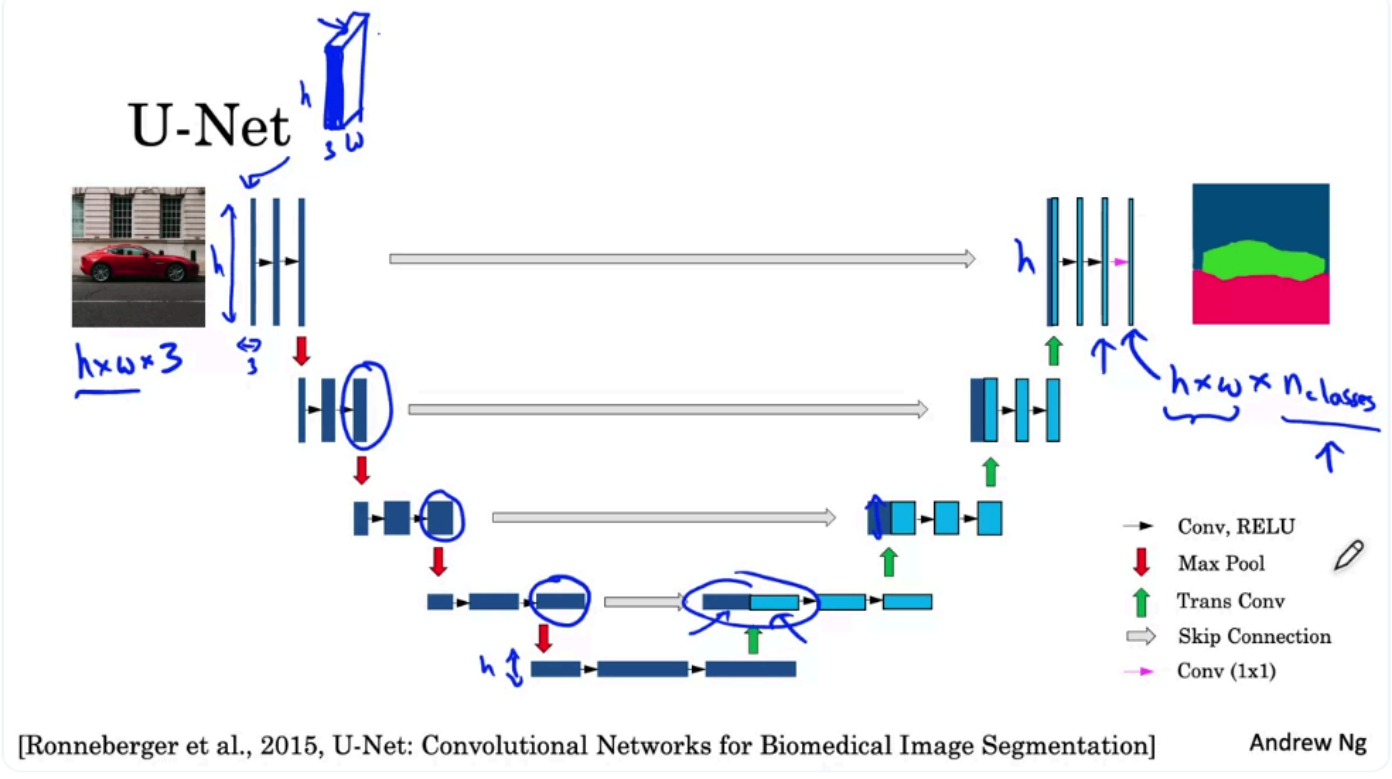
Key Takeaways

- **Encoder:** compresses the image and learns high-level semantic features.
- **Decoder:** uses transpose convolutions to recover the original resolution.
- **Skip connections:** transfer high-resolution features from the encoder directly to the decoder.
- The decoder combines:
  - **High-level contextual information** (what the object is)
  - **Low-level spatial information** (where the object boundaries are)
- This combination allows U-Net to produce accurate **pixel-wise semantic segmentation** with precise object outlines.

U-Net Architecture



U-Net Architecture



U-Net Architecture

Overview

U-Net is one of the most important architectures for **semantic segmentation**.

Its name comes from its characteristic **U-shaped structure**, consisting of:

- **Encoder (Contracting Path)**
- **Decoder (Expanding Path)**
- **Skip Connections**

```
Input
   ↓
Encoder
   ↓
Bottleneck
   ↓
Decoder
   ↓
Segmentation Map
```

Originally proposed by **Olaf Ronneberger, Philipp Fischer, and Thomas Brox** for biomedical image segmentation, U-Net is now widely used across many computer vision tasks.

Input

The input is an RGB image:

```
H × W × 3
```

where:

- `H` : image height
- `W` : image width
- `3` : RGB channels

Encoder (Contracting Path)

The encoder consists of repeated blocks of:

```
Convolution
      ↓
ReLU
      ↓
Convolution
      ↓
ReLU
      ↓
Max Pooling
```

Purpose

- Extract semantic features
- Reduce spatial resolution
- Increase the number of feature channels

Characteristics

As the network goes deeper:

Property	Change
Height	Decreases
Width	Decreases
Channels	Increase

The encoder gradually compresses the image into a compact, high-level representation.

Bottleneck

At the deepest layer:

- Feature maps have very small spatial dimensions.
- They contain rich semantic information but little precise location information.

Example:

The network may know:

> "There is a cat in the lower-right area."

but not the exact pixel boundaries.

Decoder (Expanding Path)

The decoder reconstructs the segmentation map using:

```
Transpose Convolution
        ↓
Skip Connection
        ↓
Convolution
        ↓
ReLU
```

Transpose Convolution

Used to:

- Increase height and width
- Reduce the number of channels

```
Small Feature Map
        ↓
Transpose Convolution
```

```
        ↓
Larger Feature Map
```

Skip Connections

Each decoder stage receives features directly from the corresponding encoder stage.

```
Encoder ----------------► Decoder
      Skip Connection
```

The encoder features are **copied** and concatenated with the decoder features.

Benefit

The decoder combines:

High-level semantic information

- Object identity
- Global context

with

Low-level spatial information

- Edges
- Textures
- Fine details

This produces much more accurate object boundaries.

Decoder Pipeline

Each decoder block performs:

```
Transpose Convolution
        ↓
Concatenate Skip Connection
        ↓
Convolution
        ↓
ReLU
        ↓
Convolution
        ↓
ReLU
```

This process repeats until the feature map returns to the original image resolution.

Final Layer

After the last decoder block, U-Net applies a **1 × 1 convolution**.

Purpose

Map the feature channels into class probabilities.

Output shape:

```
H × W × NumClasses
```

Example:

If there are 3 classes:

```
Output:
H × W × 3
```

If there are 10 classes:

```
Output:
H × W × 10
```

Pixel-wise Classification

For every pixel, the network predicts a probability vector:

```
[class1_score,
 class2_score,
 ...
 classN_score]
```

The predicted class is obtained by:

```
Predicted Class = argmax(probabilities)
```

Thus, every pixel is assigned exactly one semantic label.

Complete U-Net Pipeline

```
Input Image (H × W × 3)
          |
          ▼
Conv + ReLU
          |
Conv + ReLU
          |
Max Pooling
          |
          ▼
Conv + ReLU
          |
Conv + ReLU
          |
Max Pooling
          |
          ▼
      Bottleneck
          |
          ▼
Transpose Convolution
          |
Skip Connection
          |
Conv + ReLU
          |
Conv + ReLU
          |
          ▼
Transpose Convolution
          |
Skip Connection
          |
Conv + ReLU
          |
Conv + ReLU
          |
          ▼
1 × 1 Convolution
          |
          ▼
Segmentation Map (H × W × NumClasses)
```

Key Takeaways

- **U-Net** is a U-shaped encoder-decoder architecture for semantic segmentation.
- The **encoder** compresses the image using convolution and max pooling, extracting high-level semantic features.
- The **decoder** reconstructs the image resolution using transpose convolutions.
- **Skip connections** copy high-resolution encoder features directly to the decoder, preserving fine spatial details.
- A final **1 × 1 convolution** produces an **H × W × NumClasses** output.
- Each pixel is classified independently by taking the **argmax** over its class probability vector, resulting in a complete semantic segmentation map.